

GLM alapú többváltozós statisztikai modellek biztosítási alkalmazásokkal

Vakhal Péter

2026-01-01

Table of contents

Előszó	5
A könyv (tevezett) felépítése:	5
Miről nem fog szólni a kurzus és a könyv?	6
A szerzőről	6
1 Bevezetés az R környezetbe és az adatkezelésbe	7
2 Miről lesz szó a fejezetben?	8
3 Az R programnyelv és az RStudio	9
3.1 R történelem	9
3.2 RStudio - karmester az R-hez	10
3.2.1 Az RStudio felépítése	10
3.3 A workspace használata, kódok írása	11
3.4 Csomagok, package-ek telepítése, használata	12
3.5 Mesterséges intelligencia és az R	13
4 R alapok	14
4.1 Műveletek változókkal	14
4.1.1 Változó definiálása	14
4.1.2 Indexálás	15
4.1.3 Vektorizáció	17
4.2 Hiányzó adatok és kezelésük	18
4.3 Alapstatisztikák	19
4.4 Faktorok	20
4.5 Adattáblák (data frames)	20
5 Adatok importálása és adatmanipuláció	23
5.1 Adatok importálása	23
5.2 Adatmanipuláció a <code>dplyr</code> csomag segítségével	24
5.2.1 <code>select()</code>	24
5.2.2 <code>filter()</code>	27
5.2.3 <code>mutate()</code>	29
5.2.4 <code>group_by</code> és <code>summarize</code>	30
5.2.5 <code>arrange()</code>	31
5.2.6 <code>xtabs()</code> , <code>table()</code> és egyéb kereszttáblás megoldások	32

6	Feltáró adatelemzés és adatvizualizáció	33
7	Miről lesz szó a fejezetben?	34
8	Bevezetés a következtetéselméleti vizsgálatokba	35
8.1	Hipotézisek folytonos változók esetén (univariáns)	35
8.1.1	Adattípusok leíró statisztikája	36
8.1.2	Középértékek	39
8.1.3	Diszperzitási mutatók	40
8.1.4	Kvantilisok	40
8.1.5	Gyakoriságok	41
8.2	Alapvető statisztikai próbák	42
8.2.1	Hipotézisvizsgálat során elkövethető hibák	42
8.2.2	Paraméteres próbák	43
8.3	Hipotézisek diszkrét változók esetén (univariáns)	48
9	Adatvizualizáció és ggplot2 alapok	49
9.1	Eloszlások ábrázolása	49
9.2	Asszociáció ábrázolása	52
9.3	Vegyes kapcsolat ábrázolása	54
9.4	Korreláció ábrázolása	55
10	Általánosított lineáris modellek (GLM)	58
11	Miről lesz szó a fejezetben?	59
12	Általánosított lineáris modellek	60
12.1	Lineáris modellek visszatekintő	60
12.2	Modellspecifikáció	61
12.2.1	A modellek felírása	61
12.2.2	A link függvény	62
12.2.3	Idővariáns megfigyelések és exogén változók	63
12.3	Modellek folytonos eloszlású függő változóval	63
12.3.1	Normális eloszlás	63
12.3.2	Gamma eloszlás	64
12.4	Additív és multiplikatív struktúrák	65
12.5	Együtthatók becslése	66
12.5.1	Bevezetés a likelihood alapú becslésekbe	66
12.6	Regressziós koefficiensek becslése	67
12.7	Diagnosztikai mutatók	78
12.7.1	Cook-távolság	79
12.7.2	Leverage érték	79
12.8	Bináris és binomiális függő változók becslése	80
12.8.1	Probit modell	80

12.8.2	Logit modell	80
12.9	Modellek diszkrét eloszlásokra	82
12.9.1	Log-lineáris Poisson modell	82
12.10	Modell illeszkedés értékelése és a béta koefficiensek tesztelése	85
12.11	Túlszóródás	86
12.11.1	A túlszóródás okai	87
12.11.2	Kvázi-likelihood modellek	87

Előszó

Tisztelt Olvasó!

Ez az e-könyv a *Posta Biztosító* munkatársai számára készült és elsősorban a **Többváltozós statisztika** R programnyelven című tanfolyam támogatását szolgálja. A könyv részletesen tárgyalja az R programnyelv alapjait, azonban feltételezi bizonyos matematikai, statisztikai alaptudás (lineáris algebra, matematikai statisztika, mértékelmélet, valószínűségyszámítás) meglétét. A tanfolyam célja, hogy megismertesse a hallgatókat az általánosított lineáris modellek (GLM) elméletével valamint a függvényapproximáció során alkalmazott diagnosztikai eljárásokkal, különös tekintettel a biztosítási modellezés során leggyakrabban előforduló eljárásokra. A kurzus során a lineáris és frekventista valószínűségi felfogáson alapuló függvényillesztési és diagnosztikai eljárásokat tárgyaljuk, csupán az utolsó fejezet tartalmaz kitekintést a nemlineáris valamint a bayes-i esetekre.

A könyv nélkülözi a nem megkerülhetetlen matematikai levezetéseket, és elsősorban biztosítási esettanulmány alapokra épít. Az érdeklődő Olvasó azonban az ajánlott irodalomból teljes körűen tájékozódhat a tárgyalt módszerről. Az irodalmakat minden fejezet végén feltüntetjük.

Kérdés esetén állok rendelkezésükre: vakhal.peter@gmail.com

Jó munkát kívánok!

Bécs - Budapest, 2026. július 16.

Vakhal Péter

A könyv (tevezett) felépítése:

1. Bevezetés az R környezetbe és az adatkezelésbe
2. Exploratív elemzés és adatvizualizáció
3. Bevezetés a GLM elméletébe és az exponenciális eloszláscsalád alapjaiba
4. Faktorok, kontrasztok kódolása, elemzése GLM modellekben
5. Interakciók és bevezetés a nemlinearitásba GLM modellekben
6. Kárgyakoriság modellezése és a kitettség kezelése
7. Kárnagyság és a díj modellezése
8. Modellvalidáció és diagnosztika

9. Biztosítás specifikus témák (IBNR, kárfejlődési háromszög, bónusz-malusz, időfüggő kovariánsok stb.)
10. Kitekintés a GAM (nemlinearitás) és bayes-i modellekre

Miről nem fog szólni a kurzus és a könyv?

A többváltozós vagy többdimenziós statisztika egy rendkívül széles intervallumot magában foglaló tudományterület, aminek saját elméleti keretrendszere van, bár kétségtelenül sokat merít az egyváltozós módszerek tárházából. Ezeket egy kurzus alkalmával lehetetlen lenne mind átvenni, de a teljesség igénye nélkül megemlíjtük őket.

- Többdimenziós következtetéselmélet
- Dimenziócsökkentési eljárások
- Szegmentációs, partíciós eljárások - felügyelő nélkül (pl.: klaszter)
- Szegmentációs, partíciós eljárások - felügyelővel (pl.: döntési fák)
- Topologikus skálázás (pl.: MDS)
- Hálózatok, gráfok
- Többváltozós idősorok (pl.: VAR modellek)

A szerzőről

Dr. Vakhal Péter, egyetemi adjunktus. [Doktori fokozatát](#) 2022-ben szerezte a Budapesti Corvinus Egyetemen. 2011 és 2026 között az egyetem Operációkutatás és Aktuáriustudományok Tanszékének munkatársa volt, másfél évtizednyi oktatói, kutatói tapasztalattal rendelkezik többváltozós statisztika területen. 2026 augusztustól a Pannon Egyetem Kvantitatív Módszerek Intézeti Tanszék egyetemi adjunktusa. Emellett 2006-tól dolgozik kutatóként a Kopint-Tárki Konjunktúratkutató Intézetben. Egy gyermek édesapja. [Linkedin profil](#)

1 Bevezetés az R környezetbe és az adatkezelésbe

2 Miről lesz szó a fejezetben?

A fejezetben megismertetjük az olvasót az R programnyelv alapjaival valamint a legegyszerűbb adatkezelési technikáival. Fontos tudni, hogy az R nem adatkezelő szoftver, hanem egy kifejezetten statisztikai elemzésre kifejlesztett, nyílt forráskódú, tudományos közösség által fenntartott programozási nyelv. Adatkezelési lehetőségei ezért erősen korlátozottak, és bár minden végrehajtható, gyakran egyszerűbb egy táblázatkezelő szoftverben (pl.: Microsoft Excel) megoldani bizonyos műveleteket. Látni fogjuk, hogy hatványozottan igaz ez az adattisztításra, mivel az R nem rendelkezik beépített vizuális adatkezelő alkalmazással. A fejezet végén az Olvasó, kiegészítve az előadás során hallottakkal, képes lesz adatok betöltésére, azok manipulációjára, lekérdezések, kimutatások készítésére, valamint alapvető statisztikai becslések készítésére.

3 Az R programnyelv és az RStudio

3.1 R történelem

Az R programnyelv a világ egyik legelterjedtebb programozás nyelve statisztikai elemzői, kutatói körökben. Mindez elsősorban annak köszönhető, hogy a nyílt forráskódot a tudományos közösség folyamatosan fejleszti, bővíti. A nyelv fókusza minden tudományterületre kiterjed a bölcsész tudományoktól kezdve, a természettudományokon át a társadalomtudományokig. A jelenleg szabadon elérhető modulok, csomagok (*“package”* - algoritmusokat, rutinokat tartalmazó függvények) száma több tízezer a *“kínálat”* pedig folyamatosan bővül, így minden diszciplína megtalálhatja a számára releváns függvényeket.

Kis R történelem

Az R programnyelv története az 1990-es évek elejére nyúlik vissza, amikor Ross Ihaka és Robert Gentleman az Aucklandi Egyetemen megalkották az S nyelv nyílt forráskódú dialektusaként. Az elmúlt évtizedekben az R a statisztikai modellezés, az adatelemzés és a tudományos igényű vizualizáció globális, de facto standardjává nőtte ki magát. Az iparban – különösen a gyógyszerkutatásban (biostatisztika), a pénzügyi kockázatelemzésben, a makrogazdasági modellezésben és az akadémiai szférában – betöltött szerepe megkerülhetetlen, amit elsősorban a reprodukálható kutatást támogató filozófiájának és a CRAN ökoszisztéma páratlanul gazdag csomagtárának köszönhet.

Összehasonlítás más nyelvekkel

Bár a modern adattudományi projektekből gyakran merül fel a nyelvválasztás kérdése, az R markánsan elkülönül alternatíváitól:

- **R vs. Python:** Míg a Python egy általános célú szoftverfejlesztő nyelv, amely kiválóan integrálható éles produkciós környezetekbe és dominál a mély tanulás (Deep Learning) területén, addig az R-t kifejezetten adatokkal dolgozó szakemberek tervezték adatokkal dolgozó szakembereknek. Az R „statisztikai gondolkodásmódja”, az adatkeretek natív kezelése és a *tidyverse* nyújtotta elegáns adatmanipuláció páratlan hatékonyságot biztosít a feltáró elemzések során.
- **R vs. Matlab:** A Matlab kiváló, robusztus mérnöki és mátrixalapú szimulációs keretrendszer, ám zárt forráskódú és magas licencköltségekkel jár. Ezzel szemben az R teljesen ingyenes, és nyílt közösségének köszönhetően a legújabb elméleti statisztikai és ökonometria módszerek szinte azonnal elérhetővé válnak benne csom-

magok formájában.

- **R vs. C/C++:** A C-család tagjai rendkívül gyors, alacsony szintű nyelvek, amelyek optimálisak rendszerprogramozásra, de fejlesztési idejük és komplexitásuk miatt alkalmatlanok az adatok gyors, interaktív elemzésére. Ugyanakkor az R nem ellensége, hanem partnere ezeknek: a számításigényes háttérműveletek és a kritikus csomagok belső rutinjai mögött a háttérben ma is gyakran C, C++ vagy éppen Fortran kódok futnak az optimális teljesítmény érdekében.

Forrás: Gemini

3.2 RStudio - karmester az R-hez

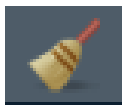
Ahogy korábban már említettük az R egy programozási nyelv, ezért az alapverzió semmilyen grafikus kezelőfelületet (*Graphical User Interface*) nem tartalmaz. A folyamatok jelentősen egyszerűsíthetők, ha a felhasználók az ingyenesen letölthető **RStudio** használják, amely független a programnyelvtől, de számos olyan rövidítést, egyszerűsítést (*“shortcut”*) tartalmaz, amely könnyen kezelhető környezetet biztosít a kódolás során. Az RStudio nem csupán az R-hez készített GUI, hanem magában integrál más programnyelveket is, mint például a Python, Julia, SQL, Stan stb, vagyis képes különböző nyelveken kódokat futtatni, az objektumok között pedig (feltételes) átjárhatóságot biztosít. Ezen kívül kezeli az úgynevezett *markdown* formátumot is, amely az elemzéseket reprodukálhatóvá teszi.

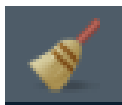
Léteznek más GUI applikációk is (pl.: **R Commander**), amelyek még kezelhetőbb felületet nyújtanak az R-hez, ugyanakkor ezek meglehetősen célzottak, vagyis csak bizonyos számú függvény futtatását teszik lehetővé (bár ezek kétségkívül a legnépszerűbbek), így nincs a lehetőségek meglehetősen leszűkítettek, használatukat csak akkor ajánljuk, ha a felhasználó meggyőződött róla, hogy a GUI-ban megtalálható függvények számára elegendők.

3.2.1 Az RStudio felépítése

Az RStudio szabadon felülete szabadon alakítható, így most csak az alapverzióban megtalálható részleteket mutatjuk be, de arra biztatjuk az Olvasót, hogy a *Tools -> Global Options* menüben szabadon fedezze fel a lehetőségeket.

Environment: itt található az elmentett adatok, függvények és objektumok. Az objektum nevére kattintva beletekinthetünk az adatok szerkezetébe, azonban függvény hívására nincs



lehetőség. Egy fontos dolog kiemelandő: a  ikonra kattintva minden változót kitöröl a munkaterületről. **A műveletet nem lehet visszavonni.**

Files: az aktuális mappa, amiben dolgozunk. Nem feltétlenül szükséges használni, de egyszerűsíti a munkát, ha látjuk a fájlstruktúrát, illetve könnyen tudunk hivatkozni rá. Kattintásra a fájlok nem hivatkozhatók.

Plots: minden vizualizáció itt jelenik meg, menthető el, törölhető ki, illetve tárolható a memóriában.

Packages: az R lelkét azok a csomagok adják, amiket a tudományos közösség fejlesztett. Itt a letöltött és telepített csomagokat láthatjuk. Újakat vagy a konzolból (lásd lentebb), vagy itt, az *Install* gombra kattintva telepíthetünk. A közösség világszerte működtet szervereket (Magyarországon az ELTE), amelyek a letölthető modulok folyamatosan tükrözött tárhelyei. Az itt megtalálható csomagok folyamatos fejlesztés alatt járnak, ellenőrzöttek. Előfordulhatnak olyan modulok is, amelyek nem találhatók meg a szerveren, ezeket külön tudjuk telepíteni. Az, hogy egy csomag nem található meg a szerveren nem feltétlenül jelenti azt, hogy nem megbízható, sokkal inkább csupán azt, hogy jelenleg nincs igazán “gazdája” (aki készítette nem kívánja karban tartani, aktualizálni stb.). Elsősorban a szerverről letöltött csomagok használatát javasoljuk.

Help: minden csomaghoz tartozik egy *help* kiegészítés is, ami kimerítően tartalmazza az összes függvény leírását, példáulval együtt.

Console: a konzol az RStudio és az R kommunikációt biztosítja, lényegében itt tud a felhasználó az R-rel közvetlenül kommunikálni. Minden itt, valamint a fenti munkaterületen kiadott utasítást az R a konzolban hatja végre és a válaszokat is elsősorban ott adja.

Workspace: ide írható, minden olyan utasítás, komment, szöveg, amit el szeretnénk magunknak tárolni, formázni stb.

3.3 A workspace használata, kódok írása

Az elemzés elsősorban a munkaterületen zajlik, a konzolba utasításokat közvetlenül nem szokás írni. Nagyon fontos, hogy a munkaterületet elsősorban szöveg írására alkalmazzuk. Bármilyen, amit ide írunk szöveggé lesz eltárolva, a csak úgy beírt kódok közvetlenül nem futtathatók, ehhez külön szükséges jelezni, hogy a következő szakaszban egy kód fog következni.



A gombra kattintva illeszthető be programkód a kiválasztott nyelven. Jellemzően minden nyelvhez elérhető fordító csomag, így nem kell több nyelven írni a jegyzetet, hanem elegendő csupán az R alapú fordítót alkalmazni (SQL nyelv esetén tipikusan ez az eljárás).

Kódokat a jegyzetfüzetből a Windows operációs rendszerben a CTRL+ENTER kombináció segítségével tudjuk futtani soronként (több sor is kijelölhető), vagy a  gombra kattintva tömbönként (a kijelölt helytől lefelé vagy fölfelé minden kódot futtasson).

Az R-ben minden, amivel dolgozunk, egy **objektum**. Statisztikai szempontból ez azt jelenti, hogy a modelljeink, az adataink és a függvényeink egyenrangúak a memóriában. Fontos megérteni a különbséget a *konzol* (ahol az utasítások azonnal végrehajthatók) és a munkaterületi *script* (ahol a reprodukálható kutatás dokumentálható) között.

3.4 Csomagok, package-ek telepítése, használata

Ahogy már fentebb említettük, az R függvényeit a tudományos közösség tartja fenn. A megírt függvényeket csomagokban tölthetjük le valamelyik ún. CRAN szerverről. Fontos, hogy letöltés után be kell tölteni a csomagot, hogy használni tudjuk. Ha egyszer letöltöttünk egy csomagot, akkor később már nem szükséges újra letölteni, de minden alkalommal be kell tölteni.

A letöltés történhet konzolból, vagy GUI-ból. Mivel nincs nagy különbség a kettő között, ezért csak a konzolos változatot mutatjuk be. Telepítsük fel a `dplyr` nevű csomagot, ami az adatmanipuláció széles körben alkalmazott modulja.

A szükséges függvények a következők:

```
install.packages("dplyr") # fontos, hogy letöltéskor mindig idézőjelbe kell tenni a csomag  
library(dplyr) # betöltéskor ugyanakkor már nem szükséges az idézőjel, hiszen a csomag már
```

Jellemzően szoktunk kapni valamilyen kommunikációt a sikeres letöltésről, telepítésről és betöltésről, bizonyos esetben akár figyelemfelhívást is.

Warning! figyelmeztetés

Csomagok betöltésekor, valamint függvények futtatásakor sok esetben kaphatunk **Warning!** üzenetet. Ezek legtöbbször nem jelentik azt, hogy a művelet ne futott le volna sikeresen, ne lenne érvényes az eredmény, de felhívja a figyelmet, hogy valamilyen feltétel sérült, nem teljesült, vagy esetleg figyelmeztet minket, hogy milyen feltételek mellett érvényesek az eredmények. Olvassuk el, és döntsük el, hogy a figyelmeztetés hogyan érinti a mi munkánkat.

Tűzfalak

Ahhoz, hogy az R le tudja tölteni a csomagokat, ki kell lépnie a tűzfalon kívülre. Ezeket a legtöbb vállalati rendszer tiltani szokta, ezért fontos, hogy az IT támogatólag lépjen fel, és adja hozzá kivételként a szoftvert. A CRAN szerver szigorúan ellenőrzött, egy csomagot akár több millióan is letölthetnek, így a víruskockázat szinte nem létezik. Ha nagyon gyorsan szükséges egy csomag, akkor azok minden esetben elérhető .zip fájlként (ilyenkor egy Google keresés vezet el a helyes CRAN vagy github oldalra). Ha

ezt a fájlt le lehet tölteni valamilyen módon, akkor ezután közvetlenül telepíthető a Tools -> Install Packages menüből, ahol az “Install from” lenyíló menüből kiválasztjuk a .zip opciót.

Sok esetben azonban ez sem járható út, többek között azért, mert egy csomag a legtöbb esetben más csomagok függvényeire is támaszkodik, amit egy `install.packages()` függvény automatikusan letölt. Ha csak annak a csomagnak a .zip fájlját töltjük le, amit használni szeretnénk, akkor elképzelhető, hogy még sok másik csomagot le kellene töltenünk, ami szinte kivitelezhetetlen.

Ilyenkor két alternatíva jön szóba, amíg az IT segít:

- Google Colab, ami egy alapvetően ingyenes felhőalapú programozási felület és képes R kódokat is futtatni, csomagokat telepíteni. Mivel ez a Google szerverén fut, ezért semmilyen engedély nem szükséges a csomagot telepítéséhez.
- RStudio Server, amit a Posit tart fenn, és egy felhőalapú alapvetően szintén ingyenes szolgáltatás.

Mindkét megoldás alapverziója ingyenes, ám rendkívül lassú, és mivel adatainkat ilyenkor egy külső szerverre helyezzük, ezért komoly adatvédelemi aggályok is felmerülnek. Mivel azonban nem vesz el számítókapacitás a saját helyi gépünktől, ezért gyakran kényelmes megoldást jelentenek.

3.5 Mesterséges intelligencia és az R

A jelenleg használatos népszerű nagy nyelvi modellek mind ismerik az R kódnyelvet és kiváló válaszokat tudnak adni a kérdésekre. Bármelyik NLP használata erősen javallt, különösen akkor, ha hosszú, repetitív elemeket tartalmazó kódot kell írunk. Jó magyarázatokat kaphatunk és a hibákat is jellemzően jól ismerik fel, de természetesen teljesen ne bízzuk rá magunkat, különösen egy olyan területen, amit kevésbé ismerünk.

4 R alapok

Az adatok beolvasása előtt tekintsük át a legfontosabb parancsokat, adatstruktúrákat, függvényeket.

4.1 Műveletek változókkal

4.1.1 Változó definiálása

A változók definiálása elsősorban a ($\leftarrow$) operátorral történik, de bizonyos esetekben alkalmazható a ($=$) operátor is. Ez utóbbit nem javasoljuk, mert kavarodást okozhat a logikai ($==$) azonosságot jelentő operátorral.

```
x<-1 # tároljuk el az értéket  
print(x) # irassuk ki az értéket
```

```
[1] 1
```

```
x # a kiíratás így is működik
```

```
[1] 1
```

```
y<-7  
  
z<-sqrt(x+y) # hivatkozzunk egy változókra a függvényben  
z
```

```
[1] 2.828427
```

Láthattuk, hogy skalárt operátor nélkül tudunk definiálni, vektor rögzítéséhez azonban a `c()` operátor alkalmazása szükséges. Mindenképpen meg kell jegyeznünk, hogy technikai értelemben az `R` nem ismeri a skalárt, hanem egy egyelemű vektorként kezeli. Ennek értelmében a helyes definiálás az `x<-c(1)` lett volna, de egy elem esetén a `c()` operátor nélkül is elfogadott.

Egy vektorban számokat és szöveget is tudunk tárolni, akár vegyesen is, de ez utóbbi esetben szöveg (string) típusként azonosítjuk a változókat. Szöveg esetén nagyon fontos, hogy a szöveg idézőjelek közé teendő. Ha nem így teszünk, akkor a beírt szöveg hivatkozásként lesz értelmezve a változókészletből.

```
x<-c(1, 2, 3.467, 4.01)
y<-c("cat", "dog")
z<-c(1, "cat", TRUE)
```

Érdemes még megemlíteni a szekvenciák (`seq()`, illetve rövidebben `from:to`) és az ismétlés függvényeket (`rep()`).

```
x_grid <- 1:10 # Egész számok 1-től 10-ig [10 elem].
p_grid <- seq(from=0, to=1, by=0.05) # Valószínűségi rács.
groups <- rep(c("A", "B"), times=5) # Ismétlődő minták generálása [10 elem].
```

4.1.2 Indexálás

A Python programozási nyelvhez képest az egyik legnagyobb különbség a vektorok indexálása. Az `R`-ben egy vektor első elemének az indexe `1`. Ezzel szemben a Python-ban az első elem indexe a `0`. A nulladik indexet az `R` nem ismeri. Egy vektor elemére vagy elemeire való hivatkozás a `változó[]` módon történik.

```
x[4] # a vektor második eleme
```

```
[1] 4.01
```

```
x[1:3] # a vektor első három eleme
```

```
[1] 1.000 2.000 3.467
```

! Változó tárolása

Amennyiben nem mentjük el a változót az `<-` operátorral, úgy az R csupán végrehajtja a parancsot, de **nem írja felül a változót**. Ügyeljünk rá, hogy felülírás esetén (például `x<-x[1:3]`) az R nem kér megerősítést.

Egy kellemes funkció, hogy a negatív indexálás az elem elhagyását jelenti a vektorból.

```
x[-1] # az első elem elhagyása a vektorból
```

```
[1] 2.000 3.467 4.010
```

```
x[-c(1,3)] # az első és a harmadik elem elhagyása a vektorból
```

```
[1] 2.00 4.01
```

```
x[-c(1:3)] # az első három elem elhagyása a vektorból
```

```
[1] 4.01
```

```
p_grid[-seq(from=2, to=21, by=2)] # minden második elem elhagyása a vektorból
```

```
[1] 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0
```

Az indexálás nagy előnye, hogy logikai operátorokat (`<==>`) is kezel.

```
x[x>3] # 3-nál nagyobb értékű elemek
```

```
[1] 3.467 4.010
```

! Logikai operátorok

R-ben az azonosságot nem “=” jellel, hanem dupla “==” jellel jelöljük. A negáció pedig “!=” operátortal jelölendő. A logikai ÉS a “&” operátorral, míg a VAGY a “|” (magyar billentyűzetten ALTGR+W) operátorral használatos. Minden esetben a vonatkozó

változót az “&” és a “|” operátorok mindkét oldalán definálni kell.

```
groups[groups=="A"] # csak "A" karakterek a vektorban
```

```
[1] "A" "A" "A" "A" "A"
```

```
p_grid[p_grid<=0.2 | p_grid>=0.8] # 0.2-nél kisebb vagy egyenlő VAGY 0.8-nál nagyobb vagy
```

```
[1] 0.00 0.05 0.10 0.15 0.20 0.80 0.85 0.90 0.95 1.00
```

```
p_grid[p_grid<0.2 & p_grid>0.8] # az eredmény egy üres vektor, mert nincs olyan szám, ami
```

```
numeric(0)
```

4.1.3 Vektorizáció

Az R-ben a vektorokkal végzett aritmetikai műveletek az elemek mentén hajtódnak végre. A “*” operátorral végzett szorzat tehát nem skaláris. Fontos, hogy csak azonos hosszúságú vektorokon végezhetők el a műveletek.

```
x <- c(1, 2, 3)
y <- c(10, 20, 30)
x * y # nem skaláris szorzat
```

```
[1] 10 40 90
```

A skaláris szorzat elvégzéséhez a “%*%” operátor használata szükséges.

```
x%*%y # skaláris szorzat
```

```
      [,1]
[1,]  140
```

! Újrafelhasználási szabály

Rendkívül fontos felhívni a figyelmet, hogy az R képes eltérő hosszúságú vektorokon is műveletek végrehajtani. Ilyenkor a rövidebb vektort ciklikusan ismétli (újrafelhasználja), amíg el nem éri a hosszabb vektor méretét. Ez a tulajdonság rendkívül hasznos például expozícióval való súlyozáskor, de veszélyes hibaforrás is lehet, ha a hosszak nem egymás többszörösei.

4.2 Hiányzó adatok és kezelésük

R-ben a hiányzó adatokat az NA operátor jelöli. Ha egy vektor hiányzó elemet tartalmaz, akkor az elemmel végrehajtott művelet automatikusan NA értéket fog kapni. Amennyiben skaláris műveletet hajtunk végre hiányzó elem mellett, akkor az eredmény vagy NA érték lesz, vagy hibaüzenet. Ez az R egy kellemes tulajdonsága, hogy minden esetben felhívja a figyelmet arra, hogy vannak hiányzó adatok.

```
x<-c(10, 20, NA, 30, 40)
y<-c(5, 15, 25, 35, 45)
x+y
```

```
[1] 15 35 NA 65 85
```

```
x*y
```

```
[1] 50 300 NA 1050 1800
```

```
x/y
```

```
[1] 2.0000000 1.3333333 NA 0.8571429 0.8888889
```

```
x%%y
```

```
      [,1]
[1,]    NA
```

4.3 Alapstatisztikák

Az R igazi ereje a statisztikai számítások esetén mutatkozik meg. A leggyakrabban használt statisztikai függvények megtalálhatók az alapverzióban, nem szükséges semmilyen csomag telepítése, betöltése. Többet között ilyen a `mean`, `sd` vagy a `summary` függvények is. Az R alapverziójában megtalálható függvények száma ezer feletti. Néhány példát áttekintünk, amelyek a leíró statisztikához szükségesek.

```
x<-rnorm(1000, mean = 0, sd = 1) # egy standard normális eloszlású 1000 elemű véletlen vek
```

💡 Alapértelmezett értékek

A legtöbb függvény különböző hiperparaméterek mellett definiálható. A legtöbb paraméternek van alapértelmezett értéke vagy függvénye, így ha az számunkra megfelelő, akkor nem kell külön definiálni. Az `rnorm` függvény például három paraméter mellett definiálható: az elemszám (`n`) az átlag (`mean`) és a szórás (`sd`), a függvénye pedig `rnorm(n, mean, sd)`. Ezek az alapértelmezett értékek $\mu = 0$ és $\sigma = 1$ (a standard normális eloszlás értékei), így ha ezek megfelelnek számunkra, akkor csupán az elemszámot kell megadni, vagyis futtatható az `rnorm(n)` függvény a paraméterek nélkül. Ha el kívánunk térni az alapértelmezett paraméterektől, akkor a módosított értéket definiálni szükséges. Az alapértelmezett értékekről mindig információt nyújt a függvény *help* melléklete, amit a `?függvény` paranccsal kérhetünk le.

Most végezzünk el néhány alapvető statisztikai becslést az előbb definiált vektorral.

```
mean(x) # várható érték
```

```
[1] 0.01978106
```

```
sd(x) # szórás
```

```
[1] 0.9862638
```

```
summary(x) # leíró statisztika
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
-2.69718	-0.61262	0.06296	0.01978	0.68663	3.57101

! Véletlen változók

Ideális esetben a véletlen változó valóban véletlen módon generálódik a processzor órajele alapján. Egy szerverre kötött gépek esetén a véletlen szám a szerver processzorának órajele alapján generálódik, így a szerver minden gépen várhatóan azonos lesz.

Fontos azonban, hogy minden futtatás új véletlen számot generál. Ha meg kívánjuk őrizni a véletlen számot és más gépeken is szeretnénk ugyanazt kapni, úgy a `set.seed()` paranccsal és egy megadott számmal ez megvalósítható. [Douglas Adams](#) után szabad a közösség gyakran a `set.seed(42)` formátumot szokta alkalmazni.

4.4 Faktorok

A biztosításmatematikai modellekben a prediktorok jelentős része kategória típusú (pl. régió, bónusz-malusz osztály). R-ben ezeket faktorként kezeljük, ami belsőleg egész számokból és a hozzájuk tartozó szintekből (levels) áll. Fontos megjegyezni, hogy a legtöbb modellben a faktorok automatikusan dummy változókká alakulnak a modellmátrix felépítésekor.

```
regioik <- c("Pest", "Vidéki város", "Pest", "Község", "Vidéki város")
regio_faktor <- factor(regioik) # az R automatikusan ábécé sorrendbe teszi a szinteket

# Gyakran szükség van egy bázisszint (referencia) rögzítésére a modellezéshez.

# ezt a levels argumentummal kontrollálhatjuk
regio_faktor <- factor(regioik, levels = c("Pest", "Vidéki város", "Község"))

# Ellenőrizzük a szinteket - a modellezésnél az első lesz a bázisszint.
levels(regio_faktor)
```

```
[1] "Pest"          "Vidéki város" "Község"
```

4.5 Adattáblák (data frames)

Az adattábla az R legfontosabb objektumtípusa. Egy lista, ahol minden elem azonos hosszúságú. Megfelel egy SQL táblának vagy Excel munkalapnak, de az oszlopok szigorúan típusosak. Nem keverendő a mátrix formátummal, ami kizárólag azonos típusú adatot tud tárolni. A legtöbb esetben az adatimportálás során eleve adattáblába történik a beolvasás. Könnyen, változónévvel vagy indexszel is hivatkozható tömbökről van szó.

Ha mi magunk hozzuk létre a táblát, akkor külön definiálandó minden oszlop (változó).

```
df <- data.frame(
  id = 1:5,
  kor = c(25, 45, 30, 55, 40),
  kar_esemeny = c(FALSE, TRUE, FALSE, FALSE, TRUE)
)
```

! Logikai értékek definálása

Az igaz-hamis értékek eleve faktorváltozóként kerülnek tárolásra. Nagyon fontos a definíálás során, hogy **mindig nagybetűvel** kell őket írni, vagy is **TRUE** vagy **FALSE**.

A data frame objektum már kellemes módon, külön ablakban jeleníthető meg, ha az *environment* ablakban az objektum nevére kattintunk.

Az adattáblák legnagyobb előnye, hogy könnyen hivatkozhatók az `\$` operátorral a változó neve után írva.

```
df$kor
```

```
[1] 25 45 30 55 40
```

Természetesen indexálással is működik, mivel azonban már nem vektorral, hanem táblával dolgozunk, ezért két indexet kell megadnunk, egy sort és egy oszlopot, ebben a sorrendben. Generikusan `variable[,]` módon írhatjuk fel a hivatkozást. Ha mindkét paramétert megadjuk, akkor értelemszerűen egy elemre, a metszéspontra hivatkozunk.

```
df[1,2] # első sor, második oszlop metszete, azaz az első elem
```

```
[1] 25
```

```
# Ha elhagyjuk valamelyik paramétert, akkor a teljes sorra/oszlopra hivatkozunk
df[,3] # minden elem (minden sor) a harmadik oszlopból
```

```
[1] FALSE TRUE FALSE FALSE TRUE
```

```
df[3,] # harmadik megfigyelés (harmadik sor) minden eleme (minden oszlopa)
```

```
id kor kar_esemeny
3 3 30 FALSE
```

```
df[2:4,] # másodiktól a negyedik sorig minden oszlop
```

```
   id kor kar_esemeny
2  2  45         TRUE
3  3  30         FALSE
4  4  55         FALSE
```

```
df[2:4,1:2] # a másodiktól a negyedik sorig, az első két oszlop
```

```
   id kor
2  2  45
3  3  30
4  4  55
```

Természetesen működik a kettő kombinációja is, de csak akkor ha egy változót indexálunk. Ilyenkor már nem adattáblánk, hanem vektorunk lesz (hiszen csak egy változóra hivatkozunk a táblából).

```
df$kor[2:3] # a kor változó második és harmadik eleme
```

```
[1] 45 30
```

5 Adatok importálása és adatmanipuláció

5.1 Adatok importálása

Az R a legtöbb adattípust tudja be tudja olvasni. Azonban saját adattípussal rendelkezik **.RData** formátumban, amennyiben R-ben dolgozunk úgy, érdemes ezt a típust alkalmazni a saját fájlok esetén és nem visszakonvertálni egy másik formátumra.

Beépített adatimportáló funkciók léteznek, ezeket érdemes használni. Mindazonáltal érdemes az adattisztítást előre elvégezni egy nagyobb rugalmasságot engedő GUI-val rendelkező szoftverben (pl: Excel), ha az úgy gyorsabb.

A betöltés a File -> Import Dataset -> From Text párbeszéd ablakon keresztül történik. Mind a “base”, mint a “readr” megoldás alkalmazható, ez utóbbi talán kényelmesebb.

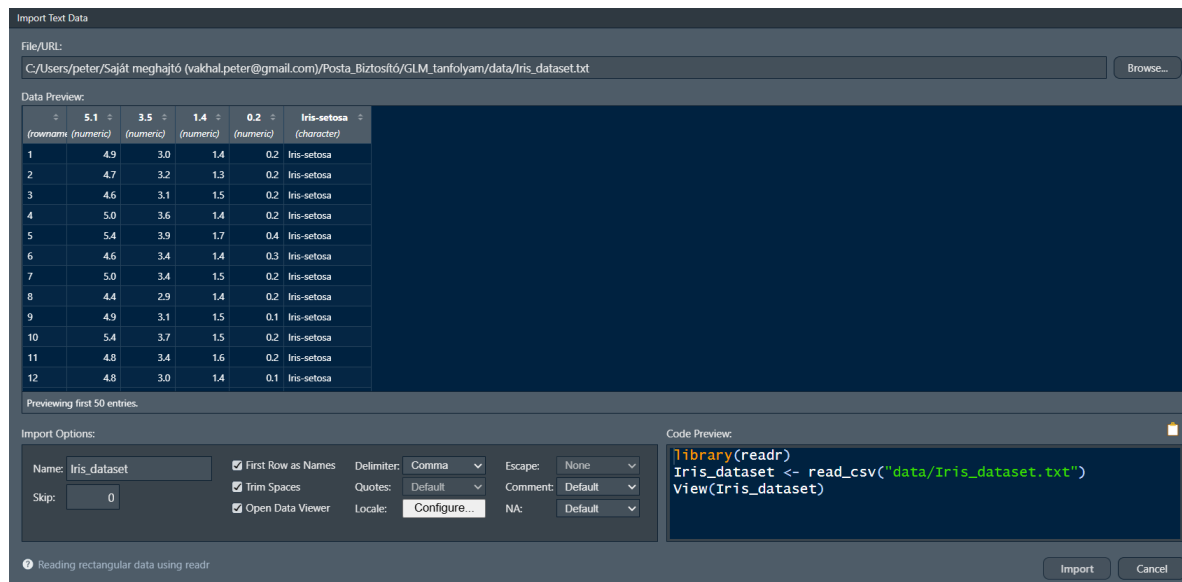


Figure 5.1: readr

A párbeszédablak magától értetődő. A beolvasás során a csomag megpróbálja felismerni az adattípust, amennyiben nem jól tette, úgy változtathatunk a változó nevére kattintva. Ügyeljünk rá, hogy gyakran előfordul, hogy külön oszlopot hoz létre a megfigyelés sorszámának. Amennyiben erre nincs szükségünk, úgy ugorjuk át ezt az oszlopot, azaz állítsuk *skip*

értékre. A jobb alsó sarokban láthatjuk a generált kódot, amit akár ki is másolhatunk, de az import gombra kattintva is végrehajtódik a függvény.

```
library(readr) # töltsük be a csomagot (előre van telepítve)
iris <- read_csv("data/Iris_dataset.txt") # olvassuk be az adatokat
```

```
Rows: 150 Columns: 5
-- Column specification -----
Delimiter: ","
chr (1): Species
dbl (4): Sepal.Length, Sepal.Width, Petal.Length, Petal.Width

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Ugyanezen az elvan működik az .xlsx (File -> Import Dataset -> From Excel) vagy a .csv (itt ugyanúgy a fentebb ismertetett From Text menüpontra van szükség) fájlok betöltése.

5.2 Adatmanipuláció a dplyr csomag segítségével

A dplyr csomag a legszélesebb körben alkalmazott adatmanipulációs csomag, amelynek szinte végtelen kombinációs lehetőségei vannak. Most a leggyakoribb műveleteket tekintjük át, de arra ösztönözzük az Olvasót, hogy szükség esetén konzultájon valamelyik NLP alapú ügynővel, hogy bonyolult manipulációkat is végre tudjon hajtani.

A dplyr csomag egyik legnagyszerűbb eleme a *pipeline* operátor, amivel objektumokat tudunk összekapcsolni. Az R ugyanis alapesetben csak a függvényen belül tudja kezelni az objektumokat, ez sokszor kényelmetlen, ha egymásba szeretnék ágyazni több függvényt is. A *pipeline* operátor a %>% módon definálható.

Nagyon fontos, hogy a dplyr csomag csak a data frame alapú objektumokat tudja kezelni!

5.2.1 select()

Ez a művelet a változók (oszlopok) egy részhalmazának kiválasztását végzi el. A változók nevét nem kell idézőjelbe tenni.

```
library(dplyr) # töltsük be a csomagot
```


Warning: a(z) 'dplyr' csomag az R 4.6.1 verziójával lett fordítva

Kapcsolódás csomaghoz: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
select(iris, Sepal.Length, Sepal.Width) # a szirm hosszá és szélesség kiválasztása
```

```
# A tibble: 150 x 2
  Sepal.Length Sepal.Width
      <dbl>         <dbl>
1         5.1         3.5
2         4.9         3
3         4.7         3.2
4         4.6         3.1
5          5         3.6
6         5.4         3.9
7         4.6         3.4
8          5         3.4
9         4.4         2.9
10        4.9         3.1
# i 140 more rows
```

```
# Ugyanez %>% operátorral
iris %>%
  select(Sepal.Length, Sepal.Width)
```

```
# A tibble: 150 x 2
  Sepal.Length Sepal.Width
      <dbl>         <dbl>
1         5.1         3.5
2         4.9         3
```

```

3          4.7          3.2
4          4.6          3.1
5           5          3.6
6          5.4          3.9
7          4.6          3.4
8           5          3.4
9          4.4          2.9
10         4.9          3.1
# i 140 more rows

```

```
select(iris, -Species, -Petal.Length) # változók kihagyása
```

```

# A tibble: 150 x 3
  Sepal.Length Sepal.Width Petal.Width
      <dbl>         <dbl>         <dbl>
1         5.1         3.5         0.2
2         4.9         3         0.2
3         4.7         3.2         0.2
4         4.6         3.1         0.2
5           5         3.6         0.2
6         5.4         3.9         0.4
7         4.6         3.4         0.3
8           5         3.4         0.2
9         4.4         2.9         0.2
10        4.9         3.1         0.1
# i 140 more rows

```

```
select(iris, contains("Petal")) # a "Petal" karakterláncot tartalmazó változónevek kiválasztása
```

```

# A tibble: 150 x 2
  Petal.Length Petal.Width
      <dbl>         <dbl>
1         1.4         0.2
2         1.4         0.2
3         1.3         0.2
4         1.5         0.2
5         1.4         0.2
6         1.7         0.4
7         1.4         0.3
8         1.5         0.2

```

```

9          1.4          0.2
10         1.5          0.1
# i 140 more rows

```

A `contains()` függvény helyett értelemszerűen alkalmazhatók a `'starts_with'`, `'ends_with'` és a `matches()` függvények is.

5.2.2 filter()

Megfigyelések (sorok) kiválasztásához egy adott kritérium alapján használjuk a `filter()` függvényt.

```
filter(iris, Species == "setosa") # csak a "setosa" fajta kiválasztás
```

```
# A tibble: 50 x 5
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
	<dbl>	<dbl>	<dbl>	<dbl>	<chr>
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa
8	5	3.4	1.5	0.2	setosa
9	4.4	2.9	1.4	0.2	setosa
10	4.9	3.1	1.5	0.1	setosa

```

# i 40 more rows

```

```

# Ugyanez %>% operátorral
iris %>%
  filter(Species=="setosa")

```

```
# A tibble: 50 x 5
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
	<dbl>	<dbl>	<dbl>	<dbl>	<chr>
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa

```

4          4.6          3.1          1.5          0.2 setosa
5          5           3.6          1.4          0.2 setosa
6          5.4          3.9          1.7          0.4 setosa
7          4.6          3.4          1.4          0.3 setosa
8          5           3.4          1.5          0.2 setosa
9          4.4          2.9          1.4          0.2 setosa
10         4.9          3.1          1.5          0.1 setosa
# i 40 more rows

```

```

filter(iris, Species == "setosa" | Species == "virginica") # "setosa" vagy "virginica" típusú

```

```

# A tibble: 100 x 5

```

```

  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
    <dbl>         <dbl>         <dbl>         <dbl> <chr>
1         5.1         3.5         1.4         0.2 setosa
2         4.9         3         1.4         0.2 setosa
3         4.7         3.2         1.3         0.2 setosa
4         4.6         3.1         1.5         0.2 setosa
5         5           3.6         1.4         0.2 setosa
6         5.4         3.9         1.7         0.4 setosa
7         4.6         3.4         1.4         0.3 setosa
8         5           3.4         1.5         0.2 setosa
9         4.4         2.9         1.4         0.2 setosa
10        4.9         3.1         1.5         0.1 setosa
# i 90 more rows

```

```

filter(iris, Species == "setosa" & Sepal.Length < 5) # 5mm-nél kisebb szirmú "setosa" típusú

```

```

# A tibble: 20 x 5

```

```

  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
    <dbl>         <dbl>         <dbl>         <dbl> <chr>
1         4.9         3         1.4         0.2 setosa
2         4.7         3.2         1.3         0.2 setosa
3         4.6         3.1         1.5         0.2 setosa
4         4.6         3.4         1.4         0.3 setosa
5         4.4         2.9         1.4         0.2 setosa
6         4.9         3.1         1.5         0.1 setosa
7         4.8         3.4         1.6         0.2 setosa
8         4.8         3         1.4         0.1 setosa
9         4.3         3         1.1         0.1 setosa

```

10	4.6	3.6	1	0.2 setosa
11	4.8	3.4	1.9	0.2 setosa
12	4.7	3.2	1.6	0.2 setosa
13	4.8	3.1	1.6	0.2 setosa
14	4.9	3.1	1.5	0.1 setosa
15	4.9	3.1	1.5	0.1 setosa
16	4.4	3	1.3	0.2 setosa
17	4.5	2.3	1.3	0.3 setosa
18	4.4	3.2	1.3	0.2 setosa
19	4.8	3	1.4	0.3 setosa
20	4.6	3.2	1.4	0.2 setosa

Több függvényt a `%>%` könnyen egymásba ágyazhatunk.

```
iris %>%
  filter(Sepal.Length < 5) %>% # 5mm-nél kisebb szírom
  select(Sepal.Width, Sepal.Length, Species) %>% # csak ezek a változók
  head(4) # csak az első négy megfigyelés kiírása
```

```
# A tibble: 4 x 3
  Sepal.Width Sepal.Length Species
      <dbl>         <dbl> <chr>
1         3         4.9 setosa
2        3.2         4.7 setosa
3        3.1         4.6 setosa
4        3.4         4.6 setosa
```

5.2.3 mutate()

A `mutate()` függvénnyel új változót adhatunk az adattáblánkhöz. Ez gyakran elő fog fordulni adatmanipuláció során, mert sok esetben segédváltozóra van szükség egy-egy bonyolultabb kimutatás előállításához. Ellátsuk elő például az értékek arányait.

```
iris %>%
  mutate(Petal.Shape = Petal.Width / Petal.Length,
         Sepal.Shape = Sepal.Width / Sepal.Length) %>%
  select(Species, Petal.Shape, Sepal.Shape)
```

```
# A tibble: 150 x 3
  Species Petal.Shape Sepal.Shape
```

	<chr>	<dbl>	<dbl>
1	setosa	0.143	0.686
2	setosa	0.143	0.612
3	setosa	0.154	0.681
4	setosa	0.133	0.674
5	setosa	0.143	0.72
6	setosa	0.235	0.722
7	setosa	0.214	0.739
8	setosa	0.133	0.68
9	setosa	0.143	0.659
10	setosa	0.0667	0.633

i 140 more rows

5.2.4 group_by és summarize

Számos adatelemzési feladat megközelíthető a split-apply-combine paradigma segítségével:

- bontsuk fel az adatokat csoportokra,
- alkalmazzunk valamilyen elemzést mindegyik csoportra,
- Kombináljuk az eredményeket.

A `dplyr` ezt nagyon egyszerűvé teszi a `group_by` és `summarize` függvények használatával. A `group_by` minden csoportot egyetlen soros összefoglalóvá csuk össze. Argumentumként azokat az oszlopneveket veszi, amelyek azokat a **kategorikus (faktor)** változókat tartalmazzák, amelyekre az összefoglaló statisztikákat ki szeretnénk számítani.

```
iris %>%
  group_by(Species) %>%
  summarise(Mean.Petal.Length = mean(Petal.Length)) # átlagos hossz kiszámítása fajonként
```

A tibble: 3 x 2

	Species	Mean.Petal.Length
	<chr>	<dbl>
1	setosa	1.46
2	versicolor	4.26
3	virginica	5.55

Több összefoglaló statisztikát is számíthatunk.

```
iris %>%
  mutate(Petal.Long = Petal.Length > 5) %>% # új változó, ami logikai érték (egy "=" jel
```

```

group_by(Species, Petal.Length) %>% # fajok és az előbb létrehozott logikai változó alap
summarise(Mean.Petal.Length = mean(Petal.Length), # csoportátlag
           n.Petals = length(Petal.Length), # elemszám
           sd.Petal.Length = sd(Petal.Length), # szórás
           SE.Petal.Length = sd(Petal.Length) / sqrt(length(Petal.Length))) # relatív h

`summarise()` has regrouped the output.
i Summaries were computed grouped by Species and Petal.Length.
i Output is grouped by Species.
i Use `summarise(.groups = "drop_last")` to silence this message.
i Use `summarise(.by = c(Species, Petal.Length))` for per-operation grouping
  (`?dplyr::dplyr_by`) instead.

# A tibble: 5 x 6
# Groups:   Species [3]
  Species Petal.Length Mean.Petal.Length n.Petals sd.Petal.Length SE.Petal.Length
  <chr>    <lgl>          <dbl>      <int>      <dbl>          <dbl>
1 setosa  FALSE           1.46        50        0.174          0.0245
2 versico~ FALSE           4.24        49        0.459          0.0655
3 versico~ TRUE            5.1         1         NA            NA
4 virgini~ FALSE           4.87         9        0.158          0.0527
5 virgini~ TRUE            5.70        41        0.489          0.0764

```

5.2.5 arrange()

Gyakran előfordul, hogy a sorokat rendezni szeretnénk csökkenő vagy növekvő sorrendben. A `dplyr` segítségével ez elegánsan elvégezhető, de természetesen alkalmazható a `sort()` függvény is, ami az alapsomag része.

```

iris %>%
  group_by(Species) %>%
  summarise(Mean.Petal.Length = mean(Petal.Length),
            n.Petals = length(Petal.Length),
            sd.Petal.Length = sd(Petal.Length),
            SE.Petal.Length = sd(Petal.Length) / sqrt(length(Petal.Length))) %>%
  arrange(-Mean.Petal.Length) # csökkenő sorrend a negatív jel miatt

# A tibble: 3 x 5
  Species Mean.Petal.Length n.Petals sd.Petal.Length SE.Petal.Length
  <chr>          <dbl>      <int>      <dbl>          <dbl>

```

1 virginica	5.55	50	0.552	0.0780
2 versicolor	4.26	50	0.470	0.0665
3 setosa	1.46	50	0.174	0.0245

5.2.6 xtabs(), table() és egyéb keresztábrás megoldások

A keresztábra készítés meglehetősen bonyolult művelet minden szoftverben, különösen, ha előtte még szükség van valamilyen adatmanipulációra. Egy keresztábrában jellemzően két faktorváltozó metszetében a gyakoriság jelenik meg. Amennyiben megvan a két faktorváltozónk úgy nem bonyolult a dolog, ha nincs, akkor elő kell állítani.

Az `xtabs()` és `table()` két előre beépített függvény, amelyek azért jók, mert későbbiekben a statisztikai függvényeket ezeket jellemzően el szokták fogadni inputként. Fontos azonban, hogy ez a kettő elemi függvény, ezért `%>%` operátorral nem mindig tudnak kapcsolódni. Legtöbbször csupán a `data=.` paraméterrel jelezni kell, hogy korábban már megadtuk az adatok forrását.

```
iris %>%
  mutate(Petal.Long = Petal.Length > 5) %>%
  xtabs(~ Species + Petal.Long, data=.)
```

	Petal.Long	
Species	FALSE	TRUE
setosa	50	0
versicolor	49	1
virginica	9	41

A `table()` az egyik legegyszerűbb belső függvény, sajnos gyakran gyorsabban eredményhez jutunk, ha külön változóként elmentjük a módosított adattáblát, majd erre hivatkozva kérjük le a keresztábrát.

```
iris2 <- iris %>%
  mutate(Petal.Long = Petal.Length > 5)

table(iris2$Species, iris2$Petal.Long)
```

	FALSE	TRUE
setosa	50	0
versicolor	49	1
virginica	9	41

6 Feltáró adatelemzés és adatvizualizáció

7 Miről lesz szó a fejezetben?

Ez a fejezet az R alapvető statisztikai eszköztárának bemutatásáról szól, amit feltáró adatelemzésként (*“Explorative Data Analysis”*) ismerünk. Mivel a mutatók a standard egyetemi statisztika oktatás részét képezik, ezért külön módszertani magyarázatot nem fűzünk az indikátorokhoz. A fejezet másik részében megismertetjük az Olvasót az adatvizualizációs lehetőségekkel a `ggplot2` nevű modul felhasználásával.

8 Bevezetés a következtetéselméleti vizsgálatokba

A következtetéselmélet célja, hogy a mintából olyan következtetéseket vonjunk le, amely segítségével a populációra vonatkozó állításokat ellenőrizhetünk. Ezeket az állításokat **hipotézisként** kell megfogalmazni, ideális esetben még a vizsgálat előtt. Univariáns esetben ez a hipotézis jellemzően a populáció egy attribútumának momentumára vonatkozik, legjellemzőbben a centrális tendenciára, vagy más néven a **várható értékre**. Ebben az esetben természetesen feltételeznünk szükséges, hogy a vizsgált változónak létezik eloszlása, még ha nem is tudjuk pontosan, csupán feltételezzük ennek az eloszlásnak a típusát. Multivariáns esetben a megfogalmazott hipotézis már a változók közötti asszociációra (például korreláció, regresszió) vonatkozik.

A megfogalmazható hipotézis alapvetően a változó típusától függ:

- Folytonos változók esetén: a momentumokra, valamint a feltételezett értéktől való eltérésre.
- Diszkrét változók esetén: amennyiben a távolság értelmezhető a megfigyelések között (például Poisson-eloszlásból származó adatok esetén), úgy a fentebb leírtak becsülhetők.
- Nominális (faktor) változók esetén: a távolság ekkor nem értelmezhető, csupán gyakoriság számolható, hipotézisvizsgálat legalább kétváltozós kiterjesztés szükséges.

8.1 Hipotézisek folytonos változók esetén (univariáns)

A hipotézis a populáció valós eloszlására vagy annak egy paraméterére vonatkozó kijelentés. Vagyis léteznek olyan $x_i \in P_0 \subseteq p$ megfigyelések, amelyekre a kijelentés igaz. A hipotézisvizsgálat ennek ellenőrzését jelenti. A következőkben ezeket tekintjük át.

Mindenekelőtt azonban töltsük be az adatbázisunkat. Az adatok forrás a [Kaggle](#) adatbázisa. A változók többsége magától értetődő, csupán két változó értelmezése definiálandó:

- DUIS (“*Driving Under Influence*”): történt-e a múltban alkohol vagy más bódító hatású készítmény által befolyásolt vezetés?
- Outcome: Volt-e benyújtott kárigénye az ügyfélnek?

```
library(readr)
car_insur <- read_csv("data/Car_Insurance_Claim.csv")
```

Rows: 10000 Columns: 19

```
-- Column specification -----
Delimiter: ","
chr  (8): AGE, GENDER, RACE, DRIVING_EXPERIENCE, EDUCATION, INCOME, VEHICLE...
dbl (11): ID, CREDIT_SCORE, VEHICLE_OWNERSHIP, MARRIED, CHILDREN, POSTAL_COD...
```

i Use ``spec()`` to retrieve the full column specification for this data.
i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

8.1.1 Adattípusok leíró statisztikája

Az adatok típusáról az `str()` függvény nyújt bővebb információt, de ugyanezekhez az adatokhoz juthatunk, ha változólistában lenyitjuk a változóhoz tartozó menüt.

```
str(car_insur)
```

```
spc_tbl_ [10,000 x 19] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ ID           : num [1:10000] 569520 750365 199901 478866 731664 ...
 $ AGE          : chr [1:10000] "65+" "16-25" "16-25" "16-25" ...
 $ GENDER       : chr [1:10000] "female" "male" "female" "male" ...
 $ RACE         : chr [1:10000] "majority" "majority" "majority" "majority" ...
 $ DRIVING_EXPERIENCE : chr [1:10000] "0-9y" "0-9y" "0-9y" "0-9y" ...
 $ EDUCATION    : chr [1:10000] "high school" "none" "high school" "university" ...
 $ INCOME       : chr [1:10000] "upper class" "poverty" "working class" "working class" ...
 $ CREDIT_SCORE : num [1:10000] 0.629 0.358 0.493 0.206 0.388 ...
 $ VEHICLE_OWNERSHIP : num [1:10000] 1 0 1 1 1 1 0 0 0 1 ...
 $ VEHICLE_YEAR  : chr [1:10000] "after 2015" "before 2015" "before 2015" "before 2015" ...
 $ MARRIED      : num [1:10000] 0 0 0 0 0 0 1 0 1 0 ...
 $ CHILDREN     : num [1:10000] 1 0 0 1 0 1 1 1 0 1 ...
 $ POSTAL_CODE  : num [1:10000] 10238 10238 10238 32765 32765 ...
 $ ANNUAL_MILEAGE : num [1:10000] 12000 16000 11000 11000 12000 13000 13000 14000 13000 ...
 $ VEHICLE_TYPE : chr [1:10000] "sedan" "sedan" "sedan" "sedan" ...
 $ SPEEDING_VIOLATIONS: num [1:10000] 0 0 0 0 2 3 7 0 0 0 ...
 $ DUIS         : num [1:10000] 0 0 0 0 0 0 0 0 0 0 ...
 $ PAST_ACCIDENTS : num [1:10000] 0 0 0 0 1 3 3 0 0 0 ...
 $ OUTCOME      : num [1:10000] 0 1 0 0 1 0 0 1 0 1 ...
- attr(*, "spec")=
```

```

.. cols(
..   ID = col_double(),
..   AGE = col_character(),
..   GENDER = col_character(),
..   RACE = col_character(),
..   DRIVING_EXPERIENCE = col_character(),
..   EDUCATION = col_character(),
..   INCOME = col_character(),
..   CREDIT_SCORE = col_double(),
..   VEHICLE_OWNERSHIP = col_double(),
..   VEHICLE_YEAR = col_character(),
..   MARRIED = col_double(),
..   CHILDREN = col_double(),
..   POSTAL_CODE = col_double(),
..   ANNUAL_MILEAGE = col_double(),
..   VEHICLE_TYPE = col_character(),
..   SPEEDING_VIOLATIONS = col_double(),
..   DUIS = col_double(),
..   PAST_ACCIDENTS = col_double(),
..   OUTCOME = col_double()
.. )
- attr(*, "problems")=<pointer: 0x000001cf21e6c410>

```

Láthatjuk, hogy numerikus és string (chr) típusú adataink vannak. Ez utóbbiakat faktorokként tudjuk kezelni. Ezekről az adatokról szeretnénk egy átfogó képet kapni. A numerikus adatokra vonatkozóan a momentumokra tudunk becsléseket kérni, a kategórikus (faktor) adatokra nézve pedig gyakoriságokat.

```
summary(car_insur)
```

ID	AGE	GENDER	RACE
Min. : 101	Length :10000	Length :10000	Length :10000
1st Qu.:249639	N.unique : 4	N.unique : 2	N.unique : 2
Median :501777	N.blank : 0	N.blank : 0	N.blank : 0
Mean :500522	Min.nchar: 3	Min.nchar: 4	Min.nchar: 8
3rd Qu.:753975	Max.nchar: 5	Max.nchar: 6	Max.nchar: 8
Max. :999976			

DRIVING_EXPERIENCE	EDUCATION	INCOME	CREDIT_SCORE
Length :10000	Length :10000	Length :10000	Min. :0.05336
N.unique : 4	N.unique : 3	N.unique : 4	1st Qu.:0.41719
N.blank : 0	N.blank : 0	N.blank : 0	Median :0.52503

Min.nchar:	4	Min.nchar:	4	Min.nchar:	7	Mean	:0.51581
Max.nchar:	6	Max.nchar:	11	Max.nchar:	13	3rd Qu.:	0.61831
						Max.	:0.96082
						NAs	:982

VEHICLE_OWNERSHIP	VEHICLE_YEAR	MARRIED	CHILDREN
Min. :0.000	Length :10000	Min. :0.0000	Min. :0.0000
1st Qu.:0.000	N.unique : 2	1st Qu.:0.0000	1st Qu.:0.0000
Median :1.000	N.blank : 0	Median :0.0000	Median :1.0000
Mean :0.697	Min.nchar: 10	Mean :0.4982	Mean :0.6888
3rd Qu.:1.000	Max.nchar: 11	3rd Qu.:1.0000	3rd Qu.:1.0000
Max. :1.000		Max. :1.0000	Max. :1.0000

POSTAL_CODE	ANNUAL_MILEAGE	VEHICLE_TYPE	SPEEDING_VIOLATIONS
Min. :10238	Min. : 2000	Length :10000	Min. : 0.000
1st Qu.:10238	1st Qu.:10000	N.unique : 2	1st Qu.: 0.000
Median :10238	Median :12000	N.blank : 0	Median : 0.000
Mean :19865	Mean :11697	Min.nchar: 5	Mean : 1.483
3rd Qu.:32765	3rd Qu.:14000	Max.nchar: 10	3rd Qu.: 2.000
Max. :92101	Max. :22000		Max. :22.000
	NAs :957		

DUIS	PAST_ACCIDENTS	OUTCOME
Min. :0.0000	Min. : 0.000	Min. :0.0000
1st Qu.:0.0000	1st Qu.: 0.000	1st Qu.:0.0000
Median :0.0000	Median : 0.000	Median :0.0000
Mean :0.2392	Mean : 1.056	Mean :0.3133
3rd Qu.:0.0000	3rd Qu.: 2.000	3rd Qu.:1.0000
Max. :6.0000	Max. :15.000	Max. :1.0000

⚠ Minden diszkrét adat kategorikus?

Sok adatbázisban előfordulnak olyan diszkrét értékek, amelyekről el kell döntenünk, hogy kategorikus vagy numerikus adatról van-e szó, és ennek megfelelő adattípust kell hozzárendelni az adatokhoz. Ha ezt elmulasztjuk, akkor fennáll annak a kockázata, hogy egy modellben nem értelmezhető eredményt fogunk kapni.

A kategorikus adatok kiválasztásánál elsősorban ezt a kérdést kell feltenni: *Értelmezhető-e távolság az adatok között?* Itt sokkal inkább szakmai szempontok bújnak meg, mintsem matematikai-statisztikai jellegűek. Amennyiben a távolság nem értelmezhető, úgy az átlag is semmitmondó.

Adatainkban például numerikus adatként van tárolva a gyorsajtások (*“speeding violations”*) száma. A többség nem követett el semmilyen vétséget (ezt a medián értékből lehet sejteni), míg vannak notórius gyorsajtók, akik egynél jóval több esetet könyvelhet-

tek el maguknak. A különbség két érték között értelmezhető, a két eset eggyel több, mint az egy és kettővel, mint a nulla. Fontos azonban megjegyezni valamit, amit az elemző csak “érez”, de matematikai szempontból “nem látszik”. Mivel a távolság értelmezhető, tudjuk, hogy akinek csak 22 gyorsajtása van (ez a maximum), az éppen 22-szer “rosszabb”, mint akinek csak egy, de csak kétszer olyan “rossz”, mint akinek 11 van. Valóban “kétszeres” a különbség két vezető között 11 és 22 gyorsajtással? Vagy igazából 4-5 gyorsajtás után már mindegy? Talán állandóan gyorsan hajt a 11-szeres és a 22-szeres is, ez utóbbit csupán többször kapták el. Érdekes lehet ebben az esetben egy új változót rögzíteni az ügyfélhez, akár egy binárisat is lehet: *notórius gyorsajtó*? Ehhez nem kényszerítjük a modellünk később, hogy különbséget tegyen két ügyfél között és mondjuk kevesebb díjat tarifáljon a 11-szeres gyorsajtónak a 22-szer gyorsajtóhoz képest. Most vizsgáljuk meg a tulajdonosi viszonyt megtestesítő (“*vehicle ownership*”) változót, amely numerikusként van kódolva, de láthatóan csak két értéket [0;1] vesz fel. Értelem-szerűen ez a változó nem numerikus, hanem kategorikus, ezért majd át kell kódolni faktorváltozóvá.

8.1.2 Középértékek

Centrális mutatók kizárólag folytonos változók esetén számíthatók, a szintaxis meglehetősen egyszerű. Használjuk a *credit score* változót a feladatra.

```
mean(car_insur$CREDIT_SCORE) # minden bizonynal tartalmaz hiányzó adatot a vektor
```

```
[1] NA
```

```
table(is.na(car_insur)) # valóban vannak hiányzó adataink
```

```
FALSE  TRUE  
188061 1939
```

```
mean(car_insur$CREDIT_SCORE, na.rm=TRUE) # hagyjuk figyelmen kívül a hiányzó adatokat
```

```
[1] 0.5158128
```

Középérték mutató még a medián, nem paraméteres próbák esetén sokszor inkább ez használatos.

```
median(car_insur$CREDIT_SCORE, na.rm=TRUE)
```

```
[1] 0.5250328
```

8.1.3 Diszperzitási mutatók

A szórás és a variancia számítása szintén rendkívül egyszerű, azonban a hiányzó adatokra itt is korrigálni szükséges.

```
sd(car_insur$CREDIT_SCORE, na.rm=TRUE) # szórás
```

```
[1] 0.1376881
```

```
var(car_insur$CREDIT_SCORE, na.rm=TRUE)
```

```
[1] 0.01895801
```

Némiképp robusztusabb szóródási mutató az interkvartilis terjedelem (Q2-Q3 közötti távolság) és az átlagos abszolút deviancia (*MAD*).

```
IQR(car_insur$CREDIT_SCORE, na.rm=TRUE)
```

```
[1] 0.2011203
```

```
mad(car_insur$CREDIT_SCORE, na.rm=TRUE)
```

```
[1] 0.1475353
```

8.1.4 Kvantilisek

A tiszta díj számításánál a kockázati prémium meghatározásához a felső kvantilisek (VaR-szerű mutatók) elemzése szükséges. A `quantile()` függvény alapértelmezetten csak a kvantilisek (Q1, Q2, Q3, Q4) számolja ki, ha eltérőt szeretnénk, akkor külön vektorban kell rögzíteni a kívánt értékeket.


```
quantile(car_insur$CREDIT_SCORE,
        probs = c(0.25, 0.5, 0.75, 0.95, 0.99),
        na.rm=TRUE) # Kvantilisek meghatározása
```

```
      25%      50%      75%      95%      99%
0.4171913 0.5250328 0.6183117 0.7226941 0.7946105
```

8.1.5 Gyakoriságok

Gyakoriságot kizárólag diszkrét vagy kategorikus változók esetén tudunk számolni a már ismert `table()` paranccsal.

```
table(car_insur$EDUCATION)
```

```
high school      none  university
      4157      1915      3928
```

A módusz számítására nincs külön függvény, mert az a gyakorisági tábla maximuma.

```
table(car_insur$SPEEDING_VIOLATIONS)
```

```
 0    1    2    3    4    5    6    7    8    9   10   11   12   13   14   15
5028 1544 1161 830 530 319 188 140 75 49 50 30 20 12 5 8
16 17 18 19 22
4 3 1 2 1
```

```
# többféle módon is számolhatjuk a móduszt
max(table(car_insur$SPEEDING_VIOLATIONS)) # ez csak a maximum értéket adja meg, de nem mon
```

```
[1] 5028
```

```
# a legegyszerűbb, ha csökkenő sorrendbe rendezzük a sort() paranccsal
```

```
sort(table(car_insur$SPEEDING_VIOLATIONS), decreasing = TRUE)
```

0	1	2	3	4	5	6	7	8	10	9	11	12	13	15	14
5028	1544	1161	830	530	319	188	140	75	50	49	30	20	12	8	5
16	17	19	18	22											
4	3	2	1	1											

```
# mivel így az első elem lesz a módusz, akár kérhetjük csak ezt az elemet
sort(table(car_insur$SPEEDING_VIOLATIONS), decreasing = TRUE)[1]
```

```
0
5028
```

8.2 Alapvető statisztikai próbák

A hipotéziseinket statisztikai próbák segítségével ellenőrizhetjük. Fontos, hogy a H_0 -ról hozott döntésünk soha nem jelent megdönthetetlen bizonyítékot, csupán valamekkora bizonyosságot (konfidencia intervallum)! Ahhoz, hogy minél bizonyosabbak legyünk a próbánk eredményét illetően *megfelelő elemszám és véletlen, azonos eloszlásból származó minta* szükséges (lásd [Glivenko-Cantelli tétel](#)). Az ideális elemszám mítosza szerint létezik olyan minta, amely esetén már biztosan megfogalmazhatjuk állításunkat. Ez részben azon is múlik, hogy milyen próbát végzünk, milyen eloszlással dolgozunk. Általánosságban azonban az az igaz, hogy az $n + 1$ elemszámú minta mindig előnyösebb, mint az n elemszámú.

8.2.1 Hipotézisvizsgálat során elkövethető hibák

A hipotézis felállítása során a vizsgálatot végző rendszerint az első fajú hiba minimalizálására törekszik (H_0 -t elvetjük, holott igaz). Emellett nem szabad elfelejteni a másodfajú hibát sem (H_0 -t elfogadjuk, holott igaz)!

A populáció bármely paraméterére irányuló becslés esetén fennáll a hiba elkövetésének kockázata. Kétféle hibát követhetünk el:

- I. fajú hiba (α): H_0 -t elutasítjuk, miközben igaz.
- II. fajú hiba (β): H_0 -t elfogadjuk, miközben hamis.

Hipotéziseket rendszerint a populáció Θ paraméterére vonatkozóan szoktunk tenni, és rendszerint az attól való feltételezett $\hat{\Theta}$ eltérést (d) vetjük statisztikai próba alá:

$$P(|\hat{\Theta} - \Theta| > d < \alpha)$$

Így az α azt a legkisebb valószínűséget jelenti, amely mellett még tévedhetünk.

8.2.2 Paraméteres próbák

8.2.2.1 Normalitás tesztek

A legtöbb próba paraméteres, azaz rendelkezünk kell valamilyen a populáció eloszlására vonatkozó feltétellel. Általában feltételezzük a normális eloszlást, erre vonatkozóan ugyanakkor nem igazán létezik megbízható próba. Az R bázisváltozata a `shapiro.test()` valamint a `ks.test` próbákat tartalmazza, amelyek elsősorban kis mintán alkalmazhatók, és nagyon érzékenyek. A `shapiro.test()` például legfeljebb 5000 megfigyelésen futtatható, ezért alkalmazzuk a `sample()` függvényt egy véletlen mintavételhez.

```
# Vizsgáljuk meg a credit_score változó normalitását  
  
shapiro.test(sample(car_insur$CREDIT_SCORE, 5000))
```

Shapiro-Wilk normality test

```
data:  sample(car_insur$CREDIT_SCORE, 5000)  
W = 0.99275, p-value = 2.076e-14
```

A Shapiro-próba null-hipotézise, hogy a vektor normális eloszlásból származik. Itt ezt el kell utasítanunk.

A Kolmogorov-Smirnov-próba már nagyobb mintán is elvégezhető, de nagyon érzékeny próbáról van szó, amely már a normalitástól való legkisebb eltérést is súlyosan bünteti. Előnye azonban, hogy szinte bármilyen eloszlást meg tudunk is vizsgálni. Mivel két *eloszlás-függvény* összehasonlítása történik, ezért a referencia eloszlásnak is kumulatív eloszlásnak kell lennie. Ezt azonban elegendő szövegesen megfogalmazni a **pnorm** paraméterrel.

```
ks.test(car_insur$CREDIT_SCORE, "pnorm") # normális eloszlás
```

Asymptotic one-sample Kolmogorov-Smirnov test

```
data:  car_insur$CREDIT_SCORE  
D = 0.57035, p-value < 2.2e-16  
alternative hypothesis: two-sided
```

Jobban szokott működni a nem paraméteres **Jarque-Bera-próba**, amely az eloszlás harmadik és negyedik momentumát hasonlítja a normális eloszlás momentumaihoz, ezáltal jóval robusztusabb eredmény érhető el. A függvény a `tseries` csomagban található. Fontos, hogy nem

szerepelhet a vektorban hiányzó adat, ezért azokat előbb az `na.omit` függvény segítségével hagyjuk figyelmen kívül.

```
library(tseries)
```

```
Registered S3 method overwritten by 'quantmod':  
  method      from  
as.zoo.data.frame zoo
```

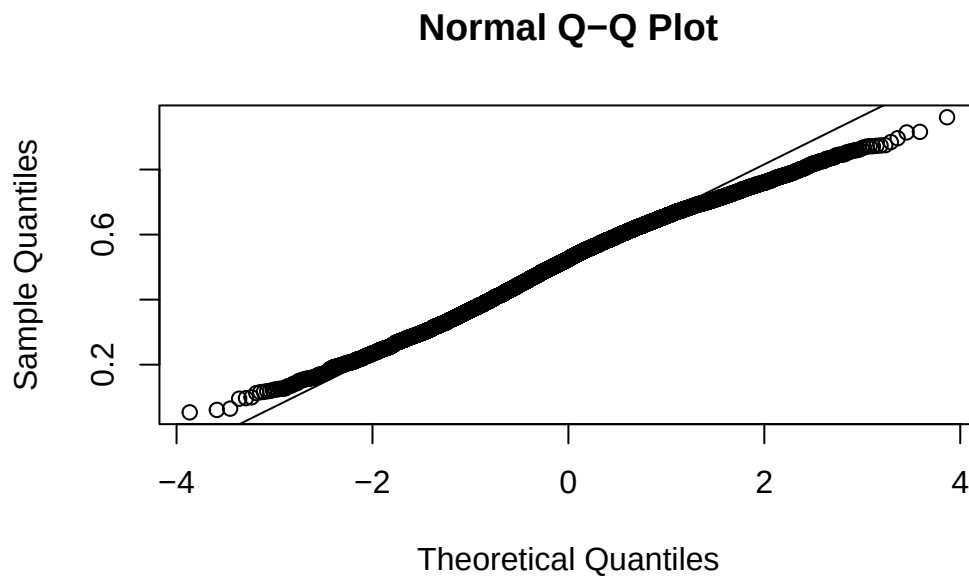
```
jarque.bera.test(na.omit(car_insur$CREDIT_SCORE))
```

Jarque Bera Test

```
data:  na.omit(car_insur$CREDIT_SCORE)  
X-squared = 139.75, df = 2, p-value < 2.2e-16
```

Sokszor nagyon hasznos az úgynevezett [QQ-plot](#) megjelenítés is, amely a tapasztalati eloszlás és a teoretikus eloszlás kvantiliseit vizsgálja. Nem tartozik hozzá semmilyen próba, a vizuális megjelenítés által szükséges döntést hozni. Ha a két eloszlás megegyezik, hogy a megfigyelések az azonossági ($x = y$) egyenesen helyezkednek el. A normális eloszlás vizsgálatára a `qqnorm()` az azonosság megjelenítésére a `qqline()` függvény alkalmazandó.

```
qqnorm(car_insur$CREDIT_SCORE)  
qqline(car_insur$CREDIT_SCORE)
```



Nincs olyan nagyon “nagy” eltérés a normális eloszlástól, inkább arról van szó, hogy mindkét végén erősen vastagodik az eloszlás széle.

i Eloszlásfüggvények (d, p, q, r család)

Az R egy egységesített eloszláskezelést folytat. Minden eloszláscsaládhoz 4 prefix tartozik, amelyekkel hivatkozhatunk az eloszlás típusára:

- **d (density)**: sűrűségfüggvény / tömegfüggvény értékek.
- **p (probability)**: kumulatív eloszlásfüggvény értékek.
- **q (quantile)**: kvantilis függvény (az eloszlásfüggvény inverze) értékek.
- **r (random)**: véletlen mintavétel.

Például kvantilis meghatározása 95%-os konfidenciaszinthez normál eloszlásnál:

```
qnorm(0.975)
```

```
[1] 1.959964
```

8.2.2.2 Várható értékre és szórásra vonatkozó próbák

Paraméteres próba lévén feltételezzük a normális eloszlást és a populáció varianciájának ismeretét. Ha mindkettő teljesül, akkor **Z-próbát** futtathatunk a $H_0 : \mu = \mu_0$ hipotézis ellenőrzésére. Ugyanakkor ez a valóságban nagyon ritkán van így, mivel a szórás rendre nem ismerjük. Ilyenkor a mintából becsüljük és **t-próbát** futtatunk. Például vizsgáljuk meg, hogy a férfiak és a nők által vezetett éves távolság. A R alapértelmezetten nem feltételezi a varianciák azonosságát, ezt azonban átállíthatjuk, ha nyomós indokunk van rá.

```
t.test(car_insur$ANNUAL_MILEAGE[car_insur$GENDER=="female"],
       car_insur$ANNUAL_MILEAGE[car_insur$GENDER=="male"],
       var.equal = FALSE)
```

Welch Two Sample t-test

```
data: car_insur$ANNUAL_MILEAGE[car_insur$GENDER == "female"] and car_insur$ANNUAL_MILEAGE[car_insur$GENDER == "male"]
t = 1.5068, df = 9040.6, p-value = 0.1319
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -26.87515 205.49922
sample estimates:
mean of x mean of y
 11741.47  11652.15
```

A t-próba alapján a két nem által vezetett éves távolság átlaga nem különbözik szignifikánsan.

Hasonló analógián tesztelhetjük a szórás azonosságára vonatkozó hipotézisünket is a `var.test` függvénnyel.

```
var.test(car_insur$ANNUAL_MILEAGE[car_insur$GENDER=="female"],
         car_insur$ANNUAL_MILEAGE[car_insur$GENDER=="male"])
```

F test to compare two variances

```
data: car_insur$ANNUAL_MILEAGE[car_insur$GENDER == "female"] and car_insur$ANNUAL_MILEAGE[car_insur$GENDER == "male"]
F = 1.0034, num df = 4540, denom df = 4501, p-value = 0.908
alternative hypothesis: true ratio of variances is not equal to 1
95 percent confidence interval:
```

```

0.9466032 1.0636909
sample estimates:
ratio of variances
1.003444

```

8.2.2.3 Arányokra vonatkozó próbák

A gyakorlatban gyakran előfordul, hogy arányokat szükséges összehasonlítani, például *két* csoport kárgyakoriságát. Fontos, hogy a próbák többsége kétutas, vagyis csak kettő csoport összehasonlítására van lehetőség. Többször összehasonlítás is lehetséges, ez azonban sokszor “kényelmetlen”, mert minden ugyanúgy egy hipotézist tudunk tesztelni, a próba azonban globális, azaz például $H_0 : \mu_1 = \mu_2 + \dots + \mu_m$.

Vizsgáljuk meg, hogy eltér-e a két nem kárgyakorisága! A `prop.test()` függvény több argumentumot kér. Nem szükséges kiszámolni az arányokat, hanem csak a “sikeres” próbálkozások számát kell megadni, valamint a mintanagyságot. A szűréshez használjuk most a `subset()` parancsot, amit nagyon hasonló a `dyplr` csomag `filter()` függvényéhez. Teszteljük a $H_0 : \mu_1 = \mu_2$ hipotézist.

```

prop.test(c(sum(subset(car_insur, car_insur$GENDER=="male")$OUTCOME), # férfiak kárgyakorisága
           sum(subset(car_insur, car_insur$GENDER=="female")$OUTCOME)), # nők kárgyakorisága
          c(nrow(subset(car_insur, car_insur$GENDER=="male")), # férfiak száma
            nrow(subset(car_insur, car_insur$GENDER=="female"))) # nők száma
)

```

2-sample test for equality of proportions with continuity correction

```

data:  c(sum(subset(car_insur, car_insur$GENDER == "male")$OUTCOME), sum(subset(car_insur, car_insur$GENDER == "female")$OUTCOME))
X-squared = 114.47, df = 1, p-value < 2.2e-16
alternative hypothesis: two.sided
95 percent confidence interval:
 0.08117319 0.11773401
sample estimates:
 prop 1      prop 2 
0.3631263 0.2636727

```

A kapott szignifikancia szint alapján bizonyosan elutasítjuk a null-hipotézist, de ez a 95%-os konfidencia intervallumból is látszik (nem tartalmazza a nullát).

8.3 Hipotézisek diszkrét változók esetén (univariáns)

A diszkrét adatok elemzése általában a kétutas keresztáblákon keresztül történik. A leggyakrabban alkalmazott próba a `chisq.test()` függvény, aminek inputja egy keresztábla a `table()` függvény segítségével jön létre. A próba azon az elven működik, hogy ha egy keresztábla peremösszegeit ismerjük, akkor a két változó *függetlenségét* feltételezve becsülhetők a cella értékei. Ekkor a becslt (E) és a tapasztalati (O) értékek közötti különbséget ($H_0 : E - O = 0$) tudjuk tesztelni. A χ^2 próba null-hipotézise:

- H_0 : a két változó között **nincs összefüggés**.
- H_1 : a két változó között **van összefüggés**.

A példánkban vizsgáljuk meg, hogy van-e összefüggés az vezetési tapasztalt és a kárgyakoriság között.

```
table(car_insur$DRIVING_EXPERIENCE,  
      car_insur$OUTCOME) # gyakorisági tábla
```

	0	1
0-9y	1313	2217
10-19y	2512	787
20-29y	2010	109
30y+	1032	20

```
chisq.test(table(car_insur$DRIVING_EXPERIENCE,  
                car_insur$OUTCOME))
```

Pearson's Chi-squared test

```
data:  table(car_insur$DRIVING_EXPERIENCE, car_insur$OUTCOME)  
X-squared = 2809.9, df = 3, p-value < 2.2e-16
```

Az eredmény alapján el kell utasítunk a null-hipotézist, és megállapíthatjuk, hogy valószínűleg van összefüggés aközött, hogy az ügyfélnek mekkora vezetési tapasztalat van és hány van-e káreseménye.

9 Adatvizualizáció és ggplot2 alapok

Az R-ben a vizualizáció szinte kizárólag a `ggplot2` elnevezésű csomagon keresztül történik. A lehetőségek tárháza szinte kimeríthetetlen, ugyanakkor kód és nem GUI alapú. A leggyakrabban használt vizualizációs lehetőségekről jó összefoglaló található az [R Graph Gallery](#) oldalon kódokkal és példákkal. Mindenképpen érdemes használni.

A `ggplot2` filozófiája szorosan összefonódik a `dplyr` csomaggal. Ugyan nem kötelező együtt használni a kettőt, de erősen ajánlott, mivel lerövidíthető, és jóval átláthatóbb kód készíthető.

A `ggplot2` “lelke” az `aes()` argumentum (“*aesthetics*”), amelyben az ábra tengelyeit, kategóriákat szükséges definiálni. Fontos, hogy az `aes()` kizárólag ezeket a beállításokat tartalmazza, ezután külön függvényben kell definiálni a diagrammtípust. `ggplot2` belül a grafikai függvényeket “+” operátor kapcsolja össze.

Természetesen alkalmazható az R saját grafikai motorja, ez azonban jóval “szegényesebb” megjelenítési lehetőségektől kímél, ráadásul jóval bonyolultabb kódolás mellett.

Egy ábrán véges dimenziószámot tudunk csak megjeleníteni. Két vagy három tengelyt tudunk definiálni, ezt színekkel kibővítve összesen 4 dimenzió megjelenítése a szokásos. Lehet ennél magasabb is, de akkor már kevésbé átlátható az ábra.

Jellemzően a következő lehetőségeink vannak:

- Egy numerikus változó esetén az *eloszlás és a sűrűség* jeleníthető meg.
- Egy diszkrét változó esetén a *gyakoriság* (tömegfüggvény) vizualizálható.
- Numerikus és numerikus változók között **korreláció** mérhető és ábrázolható.
- Kategorikus és kategorikus változók között *asszociáció* mérése és ábrázolása lehetséges.
- Kategorikus és numerikus adatok között *vegyes kapcsolat* ábrája készíthető el.

9.1 Eloszlások ábrázolása

A leggyakoribb ábrázolási lehetőség a **hisztogram** (diszkrét) és a **sűrűségfüggvény** (folytonos).

```
library(ggplot2)
require(dplyr)
```

A szükséges csomag betöltődik: dplyr

Warning: a(z) 'dplyr' csomag az R 4.6.1 verziójával lett fordítva

Kapcsolódás csomaghoz: 'dplyr'

The following objects are masked from 'package:stats':

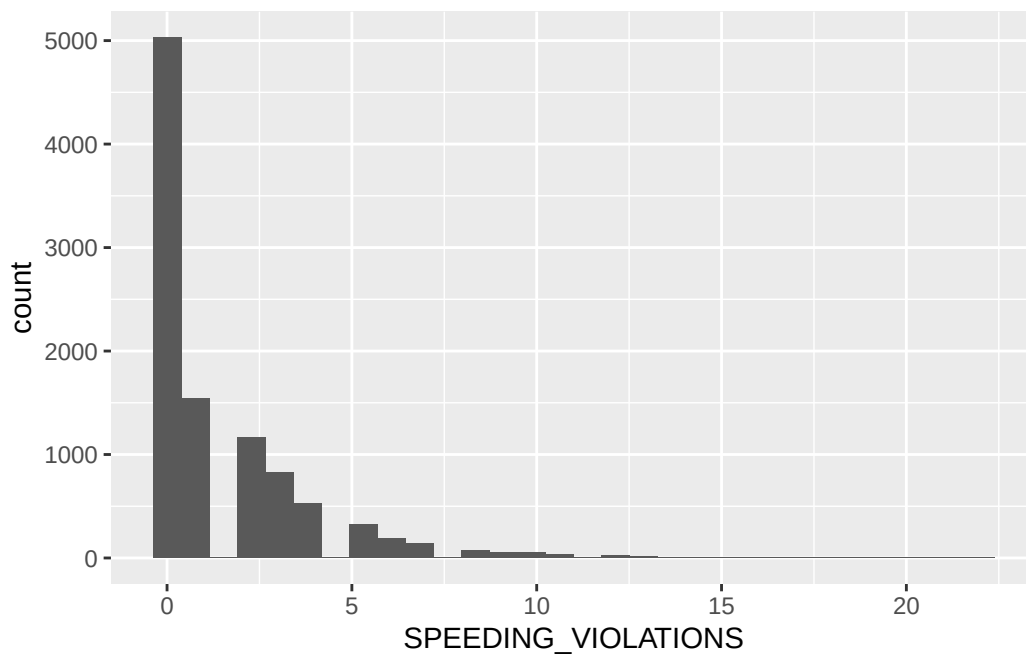
filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

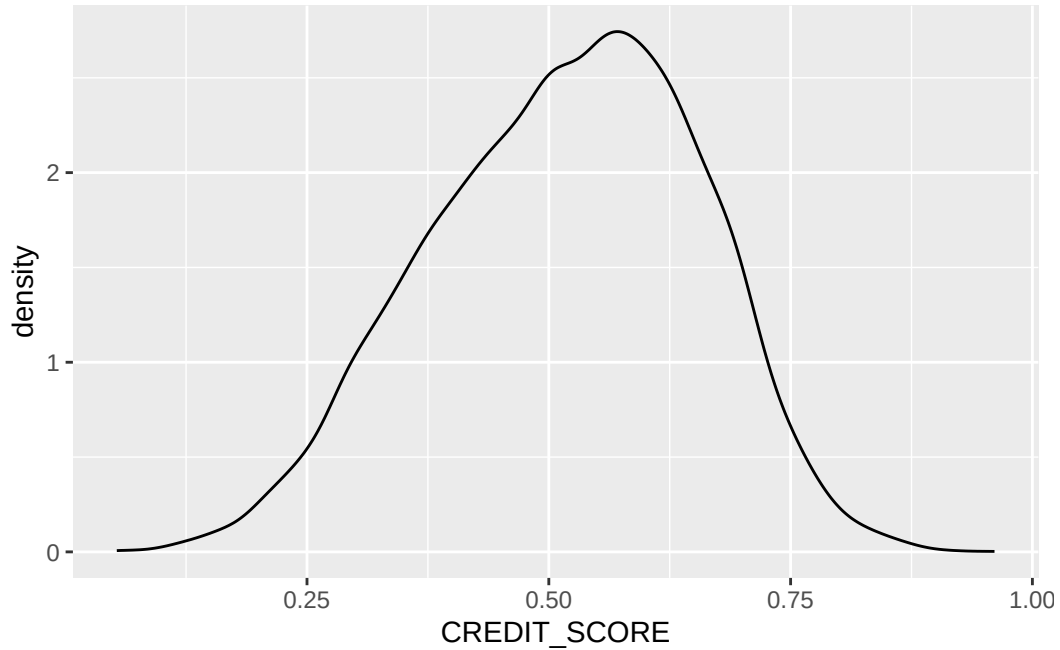
```
# hisztogramm - speedig violations  
car_insur %>%  
  ggplot(aes(SPEEDING_VIOLATIONS)) + geom_histogram()
```

`stat\_bin()` using `bins = 30`. Pick better value `binwidth`.



```
# sűrűség - credit score
car_insur %>%
  ggplot(aes(x=CREDIT_SCORE)) + geom_density()
```

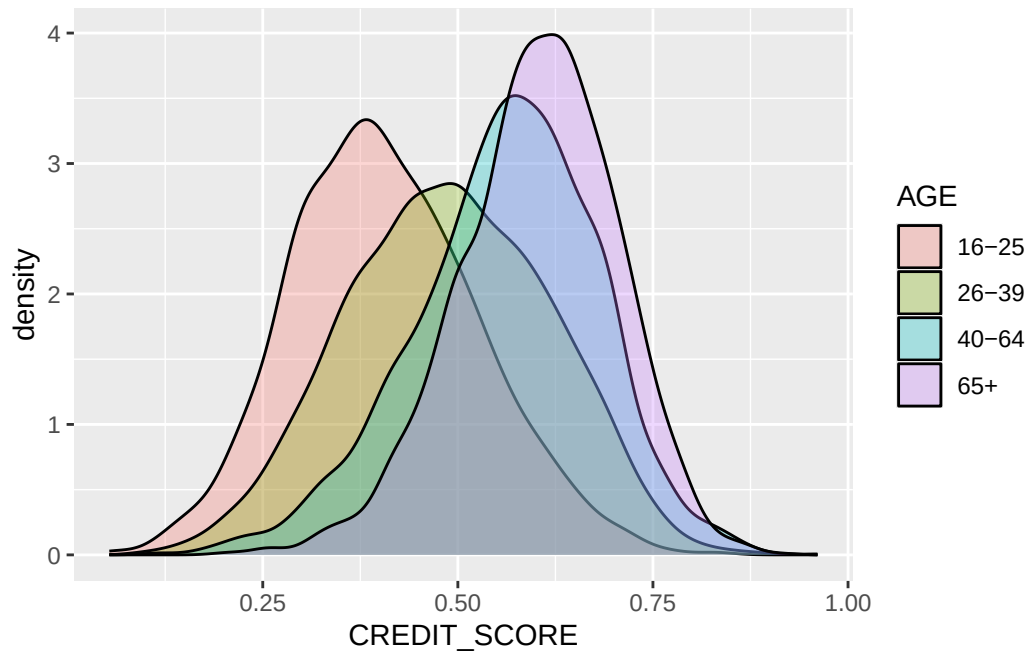
Warning: Removed 982 rows containing non-finite outside the scale range (`stat\_density()`).



A ggplot2 keretein belül mindig lehetőség van kategóriák szerinti bontásra.

```
car_insur %>%
  ggplot(aes(x=CREDIT_SCORE, group=AGE, fill = AGE)) + geom_density(alpha=0.3)
```

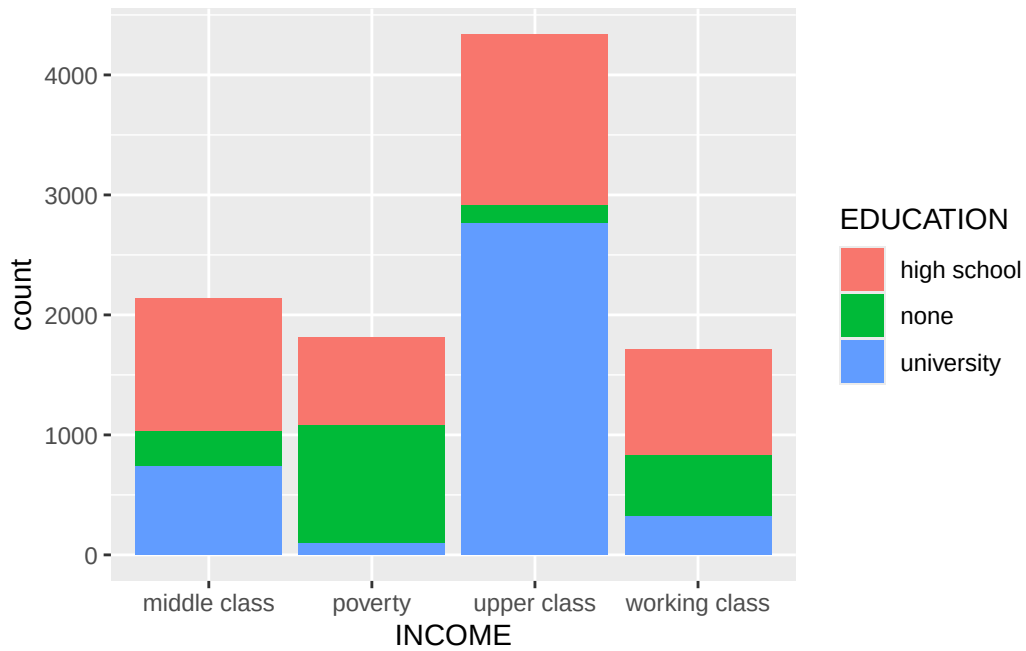
Warning: Removed 982 rows containing non-finite outside the scale range (`stat\_density()`).



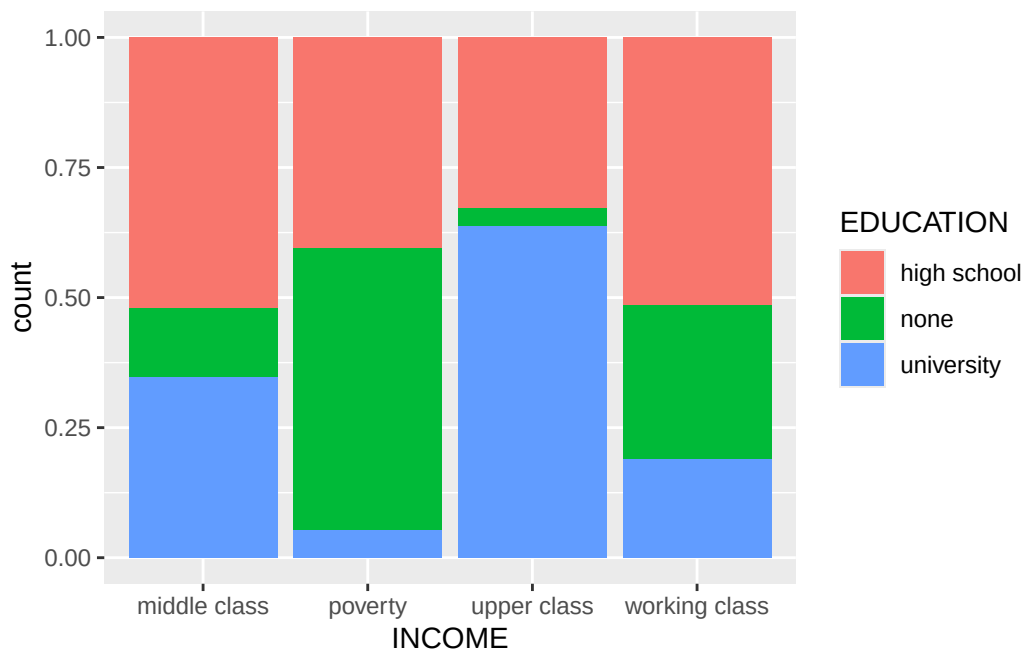
9.2 Asszociáció ábrázolása

Asszociációt két kategorikus változó között tudunk megjeleníteni úgy, hogy gyakorlatilag a gyakoriságokat tartalmazó kereszttáblát vizualizáljuk. Legyen a két változó az iskolai végzettség és az income. Vegyük észre, hogy a módszer nincs korlátozva a kategóriák száma által.

```
car_insur %>%
  ggplot(aes(x=INCOME, group=EDUCATION, fill=EDUCATION)) + geom_bar() # egyszerű oszlopdia
```



```
car_insur %>%
  ggplot(aes(x=INCOME, group=EDUCATION, fill=EDUCATION)) + geom_bar(position="fill") # 100
```



9.3 Vegyes kapcsolat ábrázolása

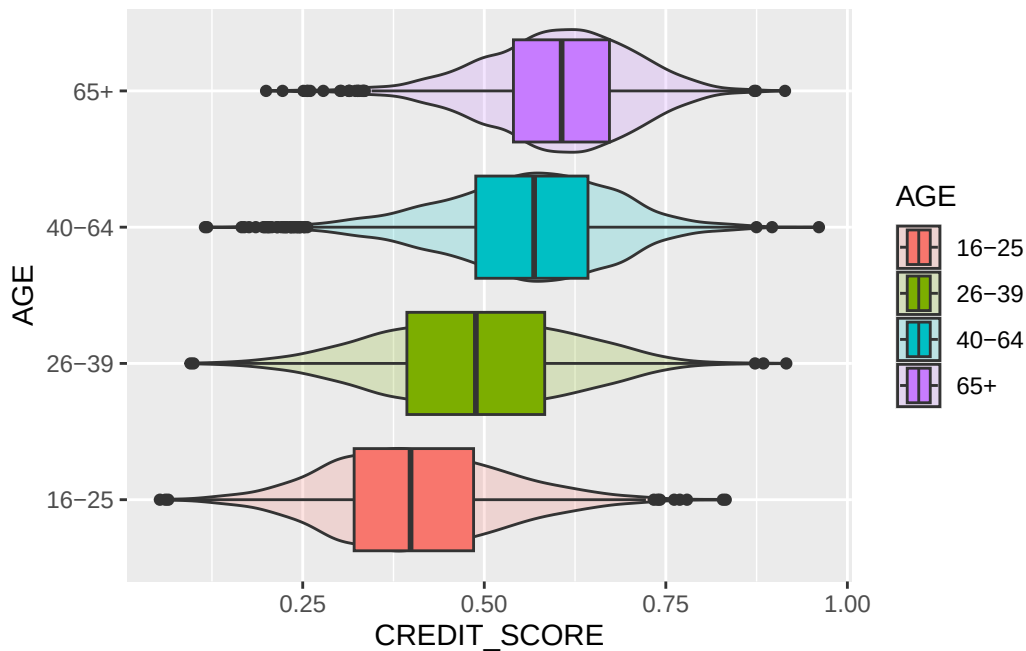
A vegyes kapcsolat vizualizációja nagyon hasonló az sűrűségfüggvények ábrázolásához, hiszen továbbra is folytonos változókat ábrázolunk, de külön kategóriánként tesszük mindezt. Az egy ábrán való megjelenít jelentősen megkönnyíti az elemzést.

Egy hasznos, több információt is megjelenítő ábra a hegedű diagramm, ami egyszerre képes elénk tárni a boxplot ábrát és a sűrűségfüggvény képét, mindezt összehasonlítható formában.

```
car_insur %>%  
  ggplot(aes(x=CREDIT_SCORE, y=AGE, fill=AGE)) + geom_violin(alpha=0.2) + geom_boxplot()
```

Warning: Removed 982 rows containing non-finite outside the scale range (`stat\_ydensity()`).

Warning: Removed 982 rows containing non-finite outside the scale range (`stat\_boxplot()`).

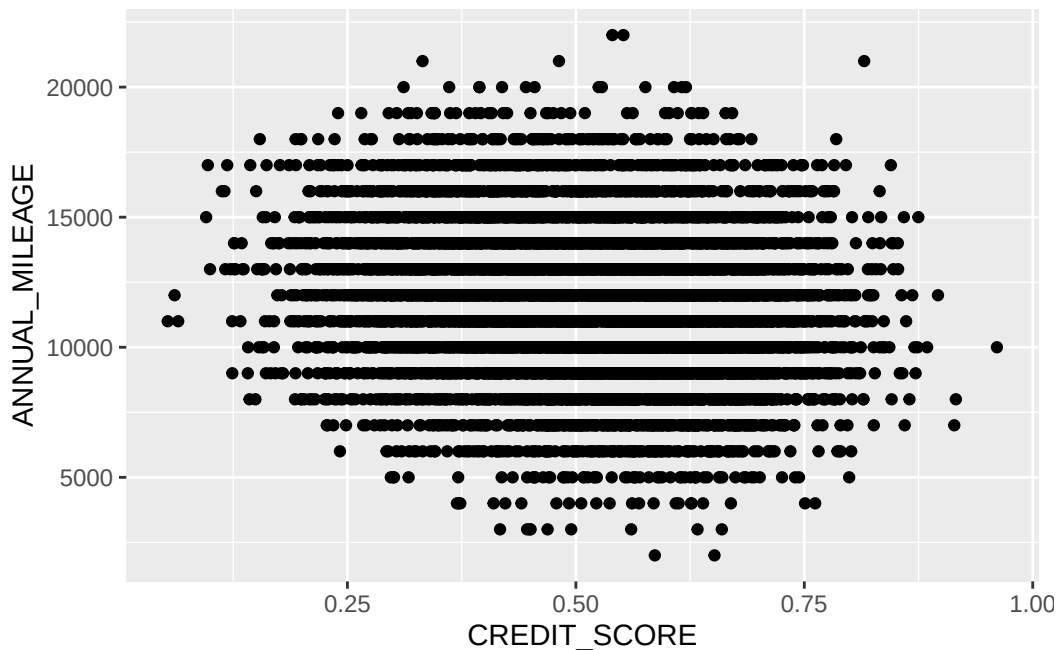


9.4 Korreláció ábrázolása

A kétváltozós ábrák legtöbbször két folytonos változó terében mutatják meg az adatok sűrűsödését. Erre több lehetőség is kínálkozik, a leggyakrabban a pontdiagramm (felhődiagramm) alkalmazott. Sajnos itt csak két valóban numerikus változónk van, amely összehasonlítása nem feltétlenül értelmes. Ábrázoljuk a *credit_score* és az *annual_milage* változókat pontdiagrammon.

```
car_insur %>%  
  ggplot(aes(x=CREDIT_SCORE, y=ANNUAL_MILEAGE)) + geom_point()
```

Warning: Removed 1851 rows containing missing values or values outside the scale range (`geom\_point()`).



Mindig van lehetőségünk valamilyen trendvonal felvételére, legyen az lineáris vagy simított, súlyozott (LOESS).

```
car_insur %>%  
  ggplot(aes(x=CREDIT_SCORE, y=ANNUAL_MILEAGE)) + geom_point() + geom_smooth(method = "lm")  
  geom_smooth(method="loess", color="blue") # simított, súlyozott
```

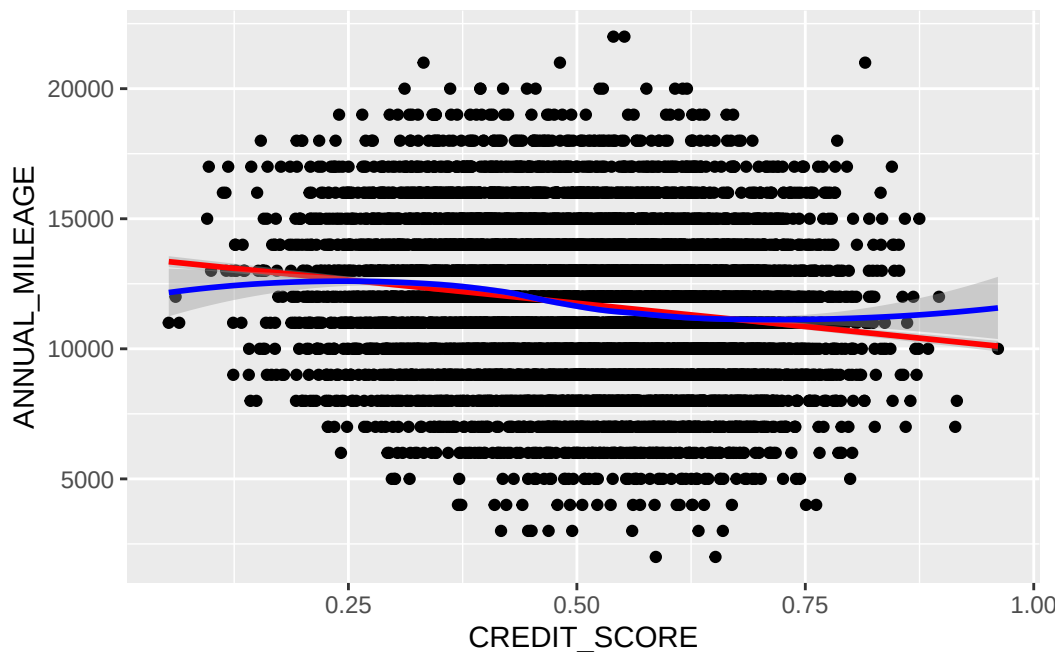
```
`geom_smooth()` using formula = 'y ~ x'
```

Warning: Removed 1851 rows containing non-finite outside the scale range
(`stat\_smooth()`).

```
`geom_smooth()` using formula = 'y ~ x'
```

Warning: Removed 1851 rows containing non-finite outside the scale range
(`stat\_smooth()`).

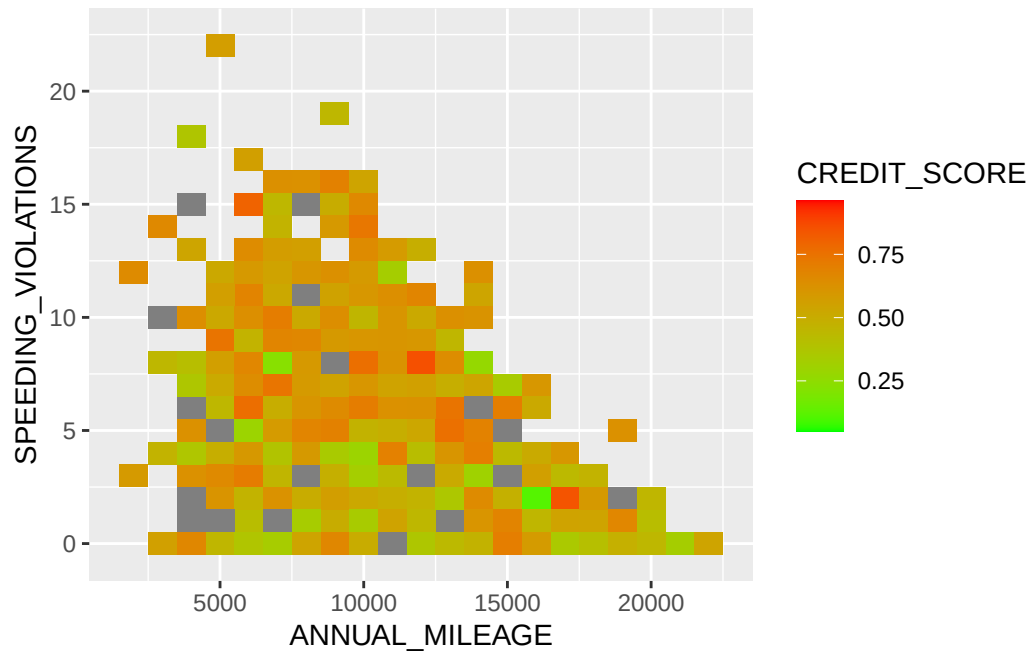
Warning: Removed 1851 rows containing missing values or values outside the scale range
(`geom\_point()`).



Néha látványosabb hőtérképen megjeleníteni az adatokat, mert így még egy dimenziót gond nélkül tudunk ábrázolni.

```
car_insur %>% ggplot(aes(x=ANNUAL_MILEAGE, y=SPEEDING_VIOLATIONS, fill=CREDIT_SCORE)) + ge
```

Warning: Removed 957 rows containing missing values or values outside the scale range
(`geom\_tile()`).



10 Általánosított lineáris modellek (GLM)

11 Miről lesz szó a fejezetben?

Ebben a fejezetben részletesen áttekintjük az általánosított lineáris modellek elméletét és gyakorlati kódolását `R`-ben. A fejezet az elméleti áttekintéssel kezdődik. Külön hangsúlyt fektetünk a biztosítási esetekre.

12 Általánosított lineáris modellek

12.1 Lineáris modellek visszatekintő

A klasszikus lineáris modellek leginkább akkor használhatók, ha a függő változó normális eloszlását megközelítőleg normálisnak feltételezzük, a folyamat pedig valamilyen lineáris struktúrával közelíthető.

Legyen tehát $y_1, y_2, \dots, y_n \in Y \sim N(\mu, \sigma_y^2)$ függő változó és legyen $x_{np} \in X^{n \times p}$ prediktor mátrix, amelyre teljesül, hogy $y_i = \beta_0 + \sum_{j=1}^p \beta_j x_{ij} + \epsilon_i$, valamint $\epsilon \sim N(0, \sigma_\epsilon^2)$. β együtthatók becslését a $\sum \epsilon^2$ hibanégyzetösszeg minimalizálása adja.

A feladat megoldásához rögzítsük, hogy $E(\epsilon) = 0$, valamint $Var(\epsilon) = \sigma_\epsilon^2 I_n$, ahol I_n egy $n \times n$ méretű *diagonális* mátrix. Célfüggvényünk a következő:

$$RSS(\beta) = \sum (y_i - \mathbf{x}_i^T \beta)^2 = (\mathbf{Y} - \mathbf{X}\beta)^T (\mathbf{Y} - \mathbf{X}\beta)$$

Átrendeződés után adódik, hogy

$$\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}$$

A megoldáshoz szükséges, hogy $(\mathbf{X}^T \mathbf{X})^{-1}$ determinánsa nem lehet nulla. Ha az így lenne, akkor tökéletes multikollinearitást jelentene. Az egyenlet megoldása numerikus módszereket igényel, azért hogy egy nagy mátrix inverzét is gyorsan ki tudja számolni a számítógép. Ha $\hat{\beta}$ vektor már rendelkezésre áll $\hat{\beta}_0$ becslés már könnyen előállítható:

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}^T \bar{\mathbf{x}}$$

Béta varianciája a következő módon becsülhető:

$$Var(\hat{\beta}) = \sigma^2 (\mathbf{X}^T \mathbf{X})^{-1}$$

ahol,

σ^2 az ϵ_i reziduumok varianciája.

Ezzel az eljárással az approximációs feladatok azon része, ahol a kezdeti feltételek teljesülnek megoldhatók, a becslések torzítatlansága mellett.

12.2 Modellspecifikáció

12.2.1 A modellek felírása

A következő feltételezések az általánosított modellekben is élnek:

1. Eloszlásokra vonatkozó feltétel: fel tennünk, hogy x_i és y_i függetlenek, és y_i eloszlása tagja az **exponenciális eloszláscsaládnak** $E(y_i|x_i) = \mu_i$ feltételes várható értékkel, és lehetőleg egy közös ϕ skálaparaméterrel.
2. Szerkezetre vonatkozó feltétel: μ_i becslése függ az $\eta_i = z_i^T \beta$ lineáris prediktortól mégpedig a következő módon: $\mu_i = h(\eta_i) = h(z_i^T \beta)$ módon. Ennek inverze pedig η_i szerint felírva pedig

$$\eta_i = g(\mu_i)$$

ahol,

h = egy simító függvény

g = a link függvény, ha h inverze

β = ismeretlen paramétervektor

z_i = magyarázó változó vektora amely egy megfelelő $z_i = z(x_i)$ transzformáció útján állítható elő.

Az általánosított lineáris modell három komponensből áll:

- **exponenciális eloszláscsalád**
- **link függvény**
- **magyarázó változók transzformált vektora**

! Az exponenciális családba tartozó eloszlások általános alakja

Minden exponenciális eloszláscsaládba tartozó eloszlás felírható a következőképpen:

$$f(y_i|\theta_i, \phi, \omega_i) = \exp \left\{ \frac{y_i \theta_i - b(\theta_i)}{\phi} + c(y_i, \phi, \omega_i) \right\}$$

ahol,

θ_i = egy ún. természetes paraméter

ϕ = skálaparaméter

$b(\cdot)$ és $c(\cdot)$ = az adott eloszláshoz tartozó specifikus paraméterek

ω_i = súly. Nem csoportosított (súlyozatlan) adatok esetén $\omega_i = 1 \ \forall i$, míg súlyozott esetben $\omega_i = n_i$, ahol n_i a megfigyelések száma az adott csoportban. Természetesen arányként is kifejezhető.

Az eloszláscsalád legfontosabb tagjai: **normális, binomiális, Poisson, gamma és inverz gamma** eloszlások.

A θ természetes paraméter a várható érték egy függvénye $\theta_i = \theta(\mu_i)$, amit az eloszlásra vonatkoztatva számítunk a következőképpen: $\mu = b'(\theta)$.

A függő változó varianciája következőképpen alakul:

$$var(y_i|x_i) = \sigma^2(\mu_i) = \phi v(\mu_i)/\omega_i$$

ahol v szinten az adott függvény specifikuma mégpedig a $v(\mu) = b''(\theta)$ formában.

Látható tehát, hogy a várható érték $\mu = h(z'\beta)$ szerkezetét minden esetben egy specifikus varianciastruktúra követi.

12.2.2 A link függvény

A megfelelő link függvényt minden esetben az eloszláshoz kell választani, azonban minden eloszláshoz rendelhető egy kanonikus alak (a lehető legegyszerűbb formában felírható alak). A természetes link függvények θ természetes paramétert kötik össze közvetlenül a lineáris prediktorral:

$$\theta = \theta(\mu) = \eta = z'\beta$$

💡 A leggyakrabban alkalmazott link függvények

Minden eloszláscsalád rendelkezik saját link függvénnyel. A leggyakoribbak:

Normális eloszlás

$$\eta = \mu$$

Poisson eloszlás

$$\eta = \log(\mu)$$

Binomiális eloszlás

$$\eta = \log\left(\frac{\mu}{1-\mu}\right)$$

Minden eloszlástípushoz megtalálható a linkfüggvény a [Wikipedia](#) oldalán.

A magyarázó változók vektorát illetően nincs igazán változás a klasszikus lineáris modellekhez képest: legtöbbször a modell tartalmaz egy konstans (β_0), ami a főátlagot jeleníti meg, tehát z a következő $z(1, \omega)$ formát ölti. Ebbe a folytonos változók közvetlenül (transzformáció nélkül) beépíthetők, míg a kategorikus változók esetében szükség lehet transzformációra, ha nem bináris változóról van szó, hanem multinominálisról (ez az ún. hot-encoding eljárás).

12.2.3 Idővariáns megfigyelések és exogén változók

A GLM modell alapértelmezetten minden megfigyelést azonos súlyúnak tekint, ami a legtöbb modell esetén nem jelent gondot. Biztosítási gyakorlatban azonban sokszor előfordul, hogy a megfigyelés (szerződés) tört évre, vagy egy évnél hosszabb időre vonatkozik. Ebben az esetben a modell nem tekintheti teljes értékűnek a megfigyelést. A töredékes megfigyelés sokszor más változókat is érint: például ha a várható káresemények száma egy év alatt 1, akkor az az ügyfél, aki csak októberben szerződik a tárgyévi káresemény száma 0,25 miközben a kockázata természetesen nem változik! A tört éveket kétféle módon vehetjük figyelembe:

- Az adatokat évesítjük, azaz leosztunk a szerződés hosszával. Gyakorisági (Poisson) modelleknél gyakran alkalmazott eljárás. Ilyenkor a kockázatviselés hosszát az ω súlyvektorban rögzítjük.
- Fix, rögzített változóként vesszük figyelembe a kockázati kitettséget (*offset*).

Árazáskor előfordulhatnak olyan tételek, amelyek a modellek kívül határozódnak meg, de a modellben figyelembe kell őket venni (exogén változók). Ilyen lehet például egy alapköltség, ami területenként eltérő és mindenképpen (rögzítetten) hozzáadódik a díjhoz. Továbbá ilyenek azok a tételek is, amelyek a díjat csökkentik, tipikusan a bonus-malus kategóriák. Ezekhez a modell nem becsül β koefficiens, de tudnia szükséges, hogy ezek a tételek még hozzáadódnak a kalkulált díjhoz. Elnevezésük **offset**.

A kitettségről és az offset-ről bővebben a 6. fejezetben lesz szó.

12.3 Modellek folytonos eloszlású függő változóval

12.3.1 Normális eloszlás

Feltételezve, hogy $y \in Y \sim N(\mu, \sigma^2)$ a *természetes link függvény* választása a klasszikus lineáris modellt implikálja:

$$\mu = \eta = z' \beta$$

Természetesen vannak esetek, mikor a nemlineáris modell jobban illeszkedik:

$$h(\eta) = \eta^2$$

$$h(\eta) = \log(\eta)$$

$$h(\eta) = \exp(\eta)$$

Ezek a nemlineáris alakok is hatékonyan kezelhetők az általánosított modell környezetében.

12.3.2 Gamma eloszlás

A gamma eloszlás egy hasznos eloszlás olyan regressziós vizsgálatok esetén, amely egy folytonos valószínűségi változót közelít, amely $\mathbb{R}^+$ tartományon értelmezett. A sűrűségfüggvény:

$$f(y) = \frac{\beta^\alpha}{\Gamma(\alpha)} e^{-\beta y} y^{\alpha-1}$$

ahol $\alpha, \beta > 0$.

Ismert, hogy a várható érték $\mu = \alpha/\beta$

A kanonikus függvény (vegyük a logaritmust, majd alakítsuk vissza):

$$f(y) = \exp\{-\beta y + (\alpha - 1)\log y + \alpha \log \beta - \log \Gamma(\alpha)\}$$

β helyére illesszük be a várható értéket, és egyszerűsítsük az első tagot.

$$f(y) = \exp\left\{(-\alpha)\frac{y}{\mu}\right\}$$

Az α paraméter ritkán ismert. Ne felejtsük el, hogy a kanonikus forma a lehető legegyszerűbb alakot takarja, és ha α ismeretlen, akkor célszerű az $\alpha = 1$ értéket használni. Ekkor

$$\theta = \eta = -\frac{1}{\mu}$$

Vagyis a helyes kanonikus link függvény az inverz.

12.4 Additív és multiplikatív struktúrák

Mint ismeretes a GLM struktúra a $g(\mu_i) = \beta_0 + \sum_{j=1}^p \beta_j x_{ij}$ formát ölti, ahol $g(\cdot)$ a link függvényt jelöli. A javasolt link függvények jó matematikai tulajdonságokkal rendelkeznek (gyors és biztosabb konvergencia az MLE becslés során), gyakorlati szempontból, különösen a biztosítási árazás során sokszor előnyösebb lehet más link függvényt alkalmazni. Ezt bizonyos keretek között szabadon megtehetjük anélkül, hogy a feltételeket megsértenénk, de vállalnunk kell annak a kockázatát, hogy az MLE becslés nem biztos, hogy sikeres lesz (jó adatstruktúra és modellspecifikáció esetén erre azonban kicsi az esély).

Az árazás során a **multiplikatív modellek** jellemzően jobban interpretálhatók, hiszen %-os változást becslünk velük így megszabadulhatunk a pénzbeli értékektől (például “értelmesebb” azt mondani, hogy az ár 10%-kal drágul, mint azt hogy 2381Ft-tal). A multiplikatív modellszerkezet link függvénye a **log link**. A *Poisson eloszlásnál* eleve ez a javasolt link függvény, míg más eloszlásoknál a **logit link** az alapértelmezett.

 **Fontos!** A log nem azonos a logit függvénnyel!

A gamma és az inverz Gauss eloszlásoknál a javasolt link függvények lecserélhetők a log link függvényre. Az említett eloszlásoknál szükséges megkötések a log link esetén is teljesülnek.

A log link ($\mathbf{X}\beta = \ln(\mu)$) és a logit függvény ($\mathbf{X}\beta = \ln\left(\frac{\mu}{1-\mu}\right)$) nem azonos, azt lecserélni nem szabad!

$$\ln(\mu) \neq \ln\left(\frac{\mu}{1-\mu}\right)$$

A logit esetén rendkívül fontos, hogy a binomiális eloszlásnak megfelelő a eredmény szigorúan 0 és 1 közé essen, ezt a log link nem teljesíti.

A gamma és az inverz gauss eloszlások esetén a javasolt link függvény lecserélhető, hiszen mindkét eloszlásnál a követelmény az, hogy $(0, +\infty)$ tartományon legyen értelmezve a függő változó, és ezt a log link is teljesíti. Ezért a multiplikatív modellszerkezet alkalmazása a legtöbb biztosítási modellezés során javasolt:

$$g(\mu_i) = \exp\left(\beta_0 + \sum_{j=1}^p \beta_j x_{ij}\right)$$

12.5 Együtthatók becslése

12.5.1 Bevezetés a likelihood alapú becslésekbe

Ismeretes, hogy minden nevezetű eloszlás legtöbbször valamilyen helyparaméterrel és skála-paraméterrel adott. Amennyiben sejtjük, hogy milyen eloszlással van dolgunk, de nem ismerjük egyetlen paraméterét sem, viszont van véletlen mindevételből származó i.i.d. mintánk, akkor becslés adható az ismeretlen paraméterek értékére.

Legyen tehát $y_1, y_2, \dots, y_n \in Y$ egy ismert eloszlásból, de ismeretlen θ paraméterrel. Ekkor a likelihood függvényünk:

$$L_Y(\theta, y) = f_y(y, \theta)$$

Vagyis ismervén a mintánkat, és sejtven a háttéreloszlást, úgy milyen eséllyel kaptuk meg azt a véletlen mintát, ami éppen a birtokunkban van?

! A likelihood függvények egy tulajdonsága

A likelihoodokat mindig a sűrűségfüggvényekből becsüljük, azonban $L_Y(\theta, y) \neq 1!$

θ paraméternek végtelen sok értéke lehetséges (ez minta konstellációról minta konstellációra változik) azonban minden lehetséges θ értékhez rendelhető egy valószínűség. Ismervén legalább egy mintát, úgy maximalizálva a valószínűségeket, megállapítható θ legvalószínűbb értéke.

Minden y_i -hez rendelhető egy egyedi $L_y(\theta, y_i)$ likelihood függvény, ami praktikusán y_i mintába kerülési esélyét mutatja meg. Ezért

$$L_Y(\theta, y) = \prod_i^n L_Y(\theta, y_i)$$

Kihasználva a logaritmus függvény monotonitásának azonosságát a log likelihood függvényt:

$$\ln L_Y(\theta, y) = \sum_i^n \ell_Y(\theta, y)$$

12.5.1.1 A score függvény

Azt a θ értéket választjuk, amely mellett a kapott mintánk a lehető legnagyobb valószínűség mellett előáll. Ez egy maximalizálási feladatot indukál, ami feltételezi a likelihood függvény differenciálhatóságát θ szerint. A likelihood függvény első deriváltja a score:

$$s_Y(\theta, y) = \frac{\partial \ell_Y}{\partial \theta}(\ell_Y(\theta, y))$$

A score várható értéke nulla, és ez a szélsőérték keresés kritériuma is egyben.

12.5.1.2 Fisher információ

Ha a likelihood függvény első deriváltjának négyzetének várható értéke az úgynevezett **Fisher információ**:

$$I_Y = E \left[\frac{\partial \ell_Y}{\partial \theta} \ell_Y(\theta, y)^2 \right]$$

Az információ megmutatja, hogy egy becslésünk θ paraméterre vonatkozóan mekkora információt tartalmaz, azaz mennyire megbízható. Ha I_Y értéke alacsony, akkor becslésünk kevésbé jó. Fontos tudni, hogy **R** az Fisher információ inverzét közli.

12.6 Regressziós koefficiensek becslése

A klasszikus lineáris regresszió megoldási módja az általánosított modellek esetén nem járható út. Ugyanakkor a likelihood elv használatával torzítatlan becslések adhatók az ismeretlen koefficiensekre.

Legyen $y_1, y_2, \dots, y_n$ függő változó és $z_1, z_2, \dots, z_n$ dizájn vektor (vagy az ezzel egyenértékű $x_1, x_2, \dots, x_n$ kovariánsok). Egy $E(y_i|x_i) = \mu_i = h(z_i'\beta)$ feltételes várható érték feladatban található β vektor maximum likelihood becslése (MLE) a likelihood vektor maximalizálásán alapul.

Először tegyük fel, hogy ϕ skálaparaméter ismert. Miután ez a paraméter a likelihood része, ezért kanonikus alakban $\phi = 1$ értékre állítható.

Az általánosított sűrűségfüggvény log-likelihood alakja a következő:

$$\ell_i(\theta_i) = \log f(y_i|\theta_i, \phi, \omega_i) = \frac{y_i\theta_i - b(\theta_i)}{\phi} \omega_i$$

Ismervén a $\theta_i = \theta(\mu_i)$ kapcsolatot, behelyettesítéssel adódik, hogy

$$\ell_i(\mu_i) = \frac{y_i \theta_i - b(\theta_i)}{\phi} \omega_i$$

Végül szintén felhasználva, hogy $\mu_i = h(z_i' \beta)$ behelyettesítéssel adódik, hogy

$$\ell_i(\beta) = \ell_i(h(z_i' \beta))$$

Mivel $y_i \in Y$ i.i.d. megfigyelések, ezért mindegyik y_i egy egyedi hozzájárulásnak tekinthető β értékéhez, amelyből következik, hogy

$$\ell(\beta) = \sum_i^n \ell_i(\beta)$$

A likelihood függvény első β szerinti parciális deriváltja adja meg a score függvényt:

$$s(\beta) = \frac{\partial \ell}{\partial \beta} = \sum_i^n s_i(\beta)$$

Az egyedi score függvény hozzájárulások pedig:

$$s_i(\beta) = z_i D_i(\beta) \sigma_i^2(\beta) [y_i - \mu_i(\beta)]$$

ahol,

$$\mu_i(\beta) = h(z_i' \beta)$$

$$\sigma_i^2 = v(h(z_i' \beta)) \phi \times \omega_i^{-1}$$

$$D_i(\beta) = \frac{\partial h(z_i' \beta)}{\partial \eta}$$

A várható *Fisher információs mátrix* pedig

$$F(\beta) = \text{cov } s(\beta) = \sum_i^n F_i(\beta)$$

Példa GLM modell gamma eloszlásra

Használjuk a *car_insur* adatbázist. Biztosítási adatokban leggyakrabban a díjak követnek Gamma eloszlást, mivel ha nem is a nullánál, de alulról mindenképpen korlátos adatokról van szó, amelyek gyakran balra erősen ferdek. Az adatbázisban sajnos nem

szerepel a kiszabott díj így készítsünk egy szintetikus gamma eloszlású díj változót.¹.

```
library(dplyr)
```

Warning: a(z) 'dplyr' csomag az R 4.6.1 verziójával lett fordítva

Kapcsolódás csomaghoz: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```

load("data/car_insur.RData")

# Beállítjuk a véletlenszám-generátort a reprodukálhatóságért
set.seed(42)

car_insur <- car_insur %>%
  mutate(
    # 1. Alapdíj (Base Rate) pl. 50 000 Ft / tetszőleges pénznem
    base_rate = 50000,

    # 2. Multiplikatív kockázati tényezők (Relatív súlyok)

    # Vezetési tapasztalat szorzója (a kezdők lényegesen többet fizetnek)
    exp_factor = case_when(
      DRIVING_EXPERIENCE == "0-9y" ~ 1.8,
      DRIVING_EXPERIENCE == "10-19y" ~ 1.3,
      DRIVING_EXPERIENCE == "20-29y" ~ 1.0,
      DRIVING_EXPERIENCE == "30y+" ~ 0.8,
      TRUE ~ 1.0
    ),

    # Évjárat szorzója (újabb autó = drágább alkatrész/javítás)
    year_factor = if_else(VEHICLE_YEAR == "after 2015", 1.25, 1.0),

    # Baleseti múlt és szabálytalanságok (minden egyes eset növeli a díjat)
    accident_factor = 1 + (PAST_ACCIDENTS * 0.15) + (SPEEDING_VIOLATIONS * 0.05),

    # Éves futásteljesítmény hatása (kisebb korrekció)
    mileage_factor = 1 + (ANNUAL_MILEAGE / 100000),

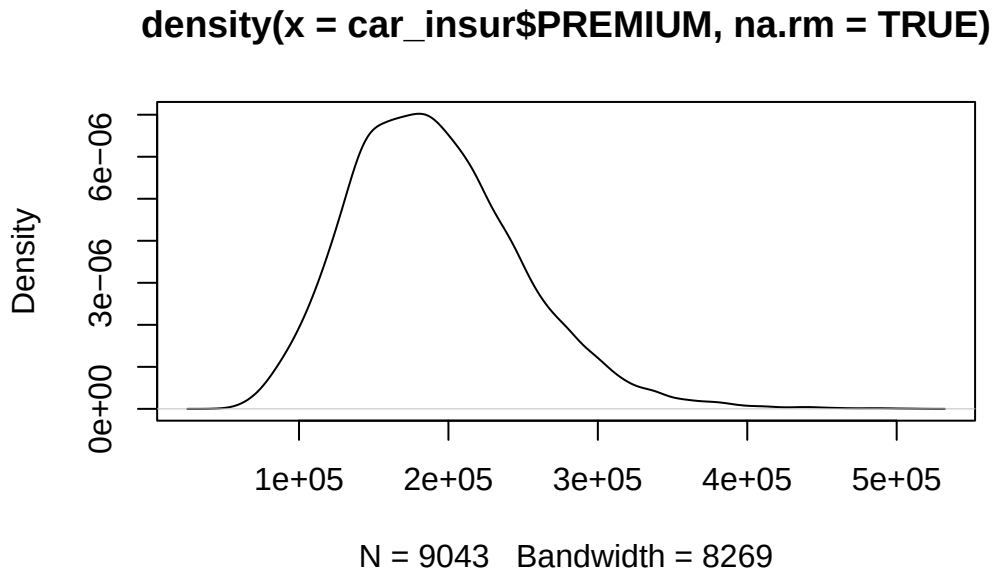
    # 3. A "valódi" elméleti várható érték (mu)
    mu_true = base_rate * exp_factor * year_factor * accident_factor * mileage_factor,

    # 4. Reális zaj hozzáadása Gamma eloszlással:
    # A Gamma(shape, scale) várható értéke = shape * scale.
    # Ha shape = 10 és scale = mu_true / 10, akkor az átlag épp mu_true lesz,
    # de lesz benne egy reális szóródás/zaj!
    PREMIUM = rgamma(n = n(), shape = 20, scale = mu_true / 10)
  ) %>%
  # Eltávolítjuk a segédváltozókat
  select(-base_rate, -exp_factor, -year_factor, -accident_factor, -mileage_factor, -mu_t

```

```
Warning: There was 1 warning in `mutate()`.
i In argument: `PREMIUM = rgamma(n = n(), shape = 20, scale = mu_true/10)`.
Caused by warning in `rgamma()`:
! NAs produced
```

```
plot(density(car_insur$PREMIUM, na.rm = TRUE))
```



A biztonság kedvéért ellenőrizzük, hogy valóban gamma eloszlásról van-e szó és becsüljük meg hozzá a paramétereit!

```
require(fitdistrplus) # szükséges csomagok
```

A szükséges csomag betöltődik: fitdistrplus

Warning: a(z) 'fitdistrplus' csomag az R 4.6.1 verziójával lett fordítva

A szükséges csomag betöltődik: MASS

Kapcsolódás csomaghoz: 'MASS'

The following object is masked from 'package:dplyr':

```
select
```

A szükséges csomag betöltődik: survival

```
require(dplyr)
```

```
premium_gamma_fit<-fitdist(na.omit(car_insur$PREMIUM), "gamma", method = "mme") # többfé
```

```
premium_normal_fit<-fitdist(na.omit(car_insur$PREMIUM), "norm", method = "mme") # csak a
```

```
summary(premium_gamma_fit) # a becsült paraméterek
```

Fitting of the distribution ' gamma ' by matching moments

Parameters :

	estimate	Std. Error
shape	1.102871e+01	1.712891e-01
rate	5.755029e-05	9.122131e-07

Loglikelihood: -111625.4 AIC: 223254.8 BIC: 223269

Correlation matrix:

	shape	rate
shape	1.0000000	0.9798426
rate	0.9798426	1.0000000

```
summary(premium_normal_fit)
```

Fitting of the distribution ' norm ' by matching moments

Parameters :

	estimate	Std. Error
mean	191635.97	606.8180
sd	57705.17	429.8687

Loglikelihood: -111970.8 AIC: 223945.6 BIC: 223959.8

Correlation matrix:

	mean	sd
mean	1.0000000000	0.0004894874
sd	0.0004894874	1.0000000000

```
gofstat(list(premium_gamma_fit, premium_normal_fit)) # illeszkedés vizsgálata a két elos
```


Goodness-of-fit statistics

	1-mme-gamma	2-mme-norm
Kolmogorov-Smirnov statistic	0.006841091	0.04243262
Cramer-von Mises statistic	0.076270283	6.31602725
Anderson-Darling statistic	0.702976946	40.49771344

Goodness-of-fit criteria

	1-mme-gamma	2-mme-norm
Akaike's Information Criterion	223254.8	223945.6
Bayesian Information Criterion	223269.0	223959.8

A gamma eloszlás feltételezését tehát elfogadhatjuk. Illesszünk GLM modellt az adatokra, a díjat magyarázzuk az összes változóval! A legfontosabb, hogy az eloszláscsalád definiálásakor **Gamma** eloszlást adjunk meg (ügyeljünk a nagy kezdőbetűre). Az alábbi eloszláscsaládok kezeltek:

- `binomial(link = "logit")`
- `gaussian(link = "identity")`
- `Gamma(link = "inverse")` vagy `Gamma(link = "log")`
- `inverse.gaussian(link = "1/mu^2")` vagy `inverse.gaussian(link = "log")`
- `poisson(link = "log")`
- `quasi(link = "identity", variance = "constant")`
- `quasibinomial(link = "logit")`
- `quasipoisson(link = "log")`

A link függvényt nem kötelező megadni, ha az alapértelmezett változat megfelelő. Bár matematikailag az inverz jelenti a kanonikus formát, az aktuáriustudományok gyakran inkább a **log** link függvényt alkalmazzák. Ennek több oka is van, amit érdemes megemlíteni:

- A biztosítási tarifa számításánál a prediktorok hatásai inkább multiplikatívak, mintsem additív formát öltenek. Ennek jellemzően gyakorlati okai vannak, mert a modell könnyebben interpretálható (x%-kal magasabb kár).
- Az inverz értékkel nem garantált a pozitív tartomány (bár nem valószínű, hogy az inverz esetén ilyen előfordulna).
- Gyorsabb algoritmus a becslés során.
- Összhangban van a többi aktuáriusi kockázati modellel.

```
require(dplyr)
glm(data = car_insur, PREMIUM~., family = Gamma(link="log")) %>% summary()
```

```
Call:
glm(formula = PREMIUM ~ ., family = Gamma(link = "log"), data = car_insur)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	1.234e+01	2.534e-02	486.942	< 2e-16 ***
ID	-1.549e-09	8.657e-09	-0.179	0.8580
AGE26-39	-8.439e-03	1.019e-02	-0.828	0.4076
AGE40-64	-7.338e-03	1.150e-02	-0.638	0.5233
AGE65+	-2.016e-03	1.342e-02	-0.150	0.8806
GENDERmale	7.200e-03	5.576e-03	1.291	0.1967
RACEminority	-1.136e-02	8.404e-03	-1.351	0.1766
DRIVING_EXPERIENCE10-19y	-2.694e-01	8.896e-03	-30.288	< 2e-16 ***
DRIVING_EXPERIENCE20-29y	-5.247e-01	1.155e-02	-45.432	< 2e-16 ***
DRIVING_EXPERIENCE30y+	-7.425e-01	1.583e-02	-46.905	< 2e-16 ***
EDUCATIONnone	1.579e-02	7.625e-03	2.071	0.0384 *
EDUCATIONuniversity	2.203e-03	6.098e-03	0.361	0.7180
INCOMEpoverty	-1.438e-02	1.118e-02	-1.286	0.1984
INCOMEupper class	2.460e-03	8.146e-03	0.302	0.7626
INCOMeworking class	-5.451e-03	8.998e-03	-0.606	0.5447
CREDIT_SCORE	-2.329e-03	2.778e-02	-0.084	0.9332
VEHICLE_OWNERSHIP	7.827e-03	6.328e-03	1.237	0.2161
VEHICLE_YEARbefore 2015	-2.218e-01	5.969e-03	-37.158	< 2e-16 ***
MARRIED	-1.975e-03	6.075e-03	-0.325	0.7452
CHILDREN	-9.619e-04	6.428e-03	-0.150	0.8811
POSTAL_CODE	1.045e-07	1.400e-07	0.746	0.4555
ANNUAL_MILEAGE	7.446e-06	1.144e-06	6.506	8.17e-11 ***
VEHICLE_TYPEsports car	-5.641e-03	1.183e-02	-0.477	0.6335
SPEEDING_VIOLATIONS	2.975e-02	1.601e-03	18.585	< 2e-16 ***
DUIS	5.510e-03	5.004e-03	1.101	0.2709
PAST_ACCIDENTS	9.686e-02	2.021e-03	47.940	< 2e-16 ***
OUTCOME	-1.055e-03	7.225e-03	-0.146	0.8839

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for Gamma family taken to be 0.05157602)

Null deviance: 737.71 on 8148 degrees of freedom
Residual deviance: 422.33 on 8122 degrees of freedom
(1851 observations deleted due to missingness)
AIC: 196627

Number of Fisher Scoring iterations: 4

Sok változónk van, amelyek nem járulnak hozzá a becsléshez, ezért ezek a β_i nulla értéket vesznek fel, de legalábbis a $H_0 : \beta_i = 0$ hipotézist nem lehet elutasítani. A legegyszerűbb módja annak, hogy csak a szignifikáns prediktorok maradjanak a modellben az, hogy valamilyen változószelekciós eljárást alkalmazunk. A legegyszerűbb (és egyben legdurvább) ilyen módszer a `step()` (*stepwise*) algoritmus használata.

```
library(MASS)
backward_gamma<-stepAIC(glm(data = na.omit(car_insur), PREMIUM~., family = Gamma(link="log"))
summary(backward_gamma)
```

Call:

```
glm(formula = PREMIUM ~ DRIVING_EXPERIENCE + VEHICLE_OWNERSHIP +
    VEHICLE_YEAR + ANNUAL_MILEAGE + SPEEDING_VIOLATIONS + PAST_ACCIDENTS,
    family = Gamma(link = "log"), data = na.omit(car_insur))
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	1.233e+01	1.359e-02	907.47	< 2e-16 ***
DRIVING_EXPERIENCE10-19y	-2.725e-01	6.636e-03	-41.06	< 2e-16 ***
DRIVING_EXPERIENCE20-29y	-5.254e-01	8.765e-03	-59.95	< 2e-16 ***
DRIVING_EXPERIENCE30y+	-7.396e-01	1.227e-02	-60.29	< 2e-16 ***
VEHICLE_OWNERSHIP	1.017e-02	5.652e-03	1.80	0.0719 .
VEHICLE_YEARbefore 2015	-2.233e-01	5.573e-03	-40.07	< 2e-16 ***
ANNUAL_MILEAGE	7.529e-06	9.470e-07	7.95	2.11e-15 ***
SPEEDING_VIOLATIONS	3.070e-02	1.517e-03	20.23	< 2e-16 ***
PAST_ACCIDENTS	9.733e-02	1.910e-03	50.95	< 2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

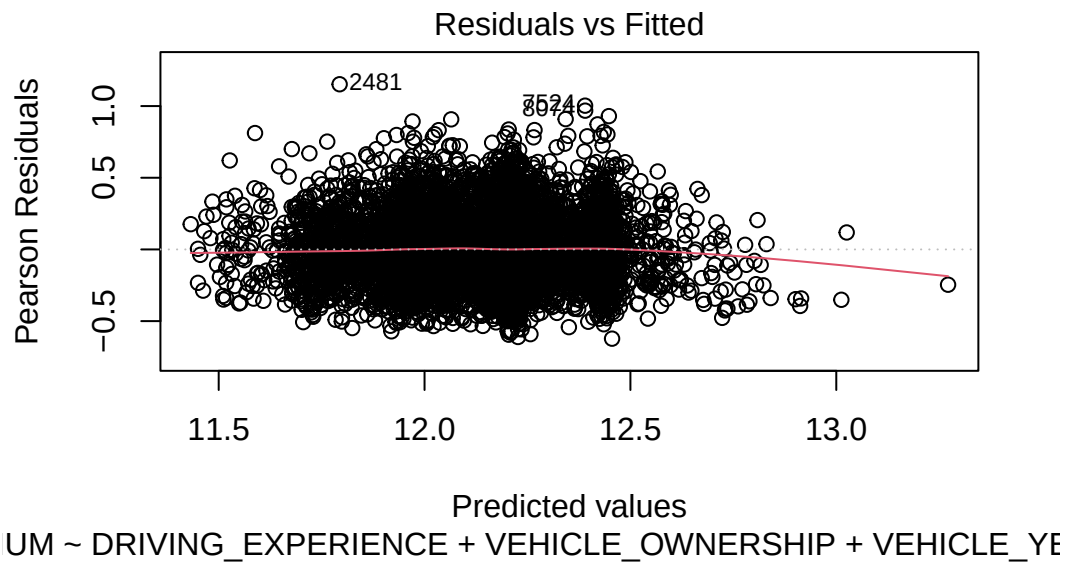
(Dispersion parameter for Gamma family taken to be 0.05153955)

Null deviance: 737.71 on 8148 degrees of freedom
Residual deviance: 422.93 on 8140 degrees of freedom
AIC: 196603

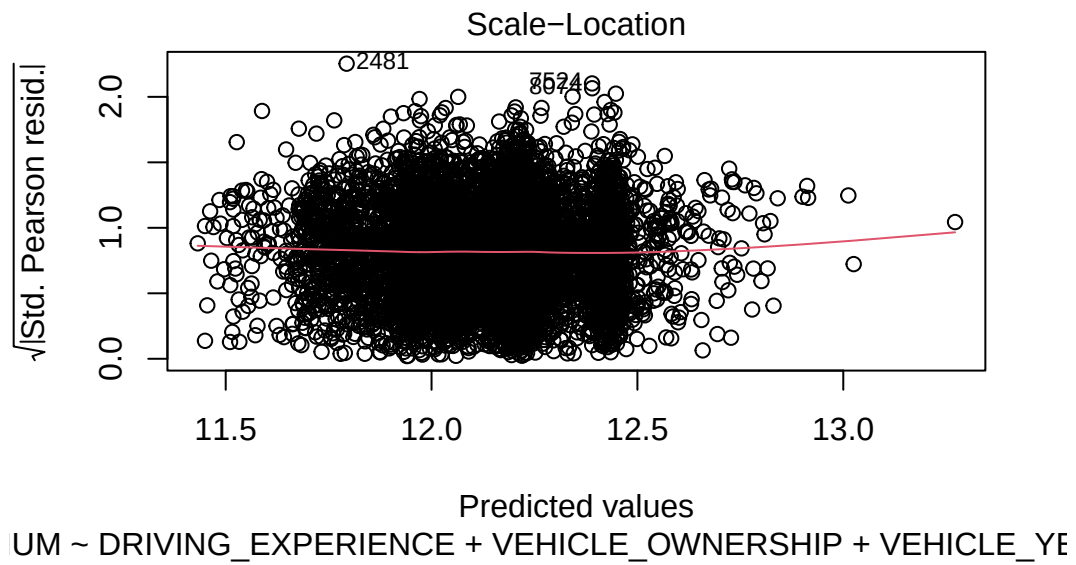
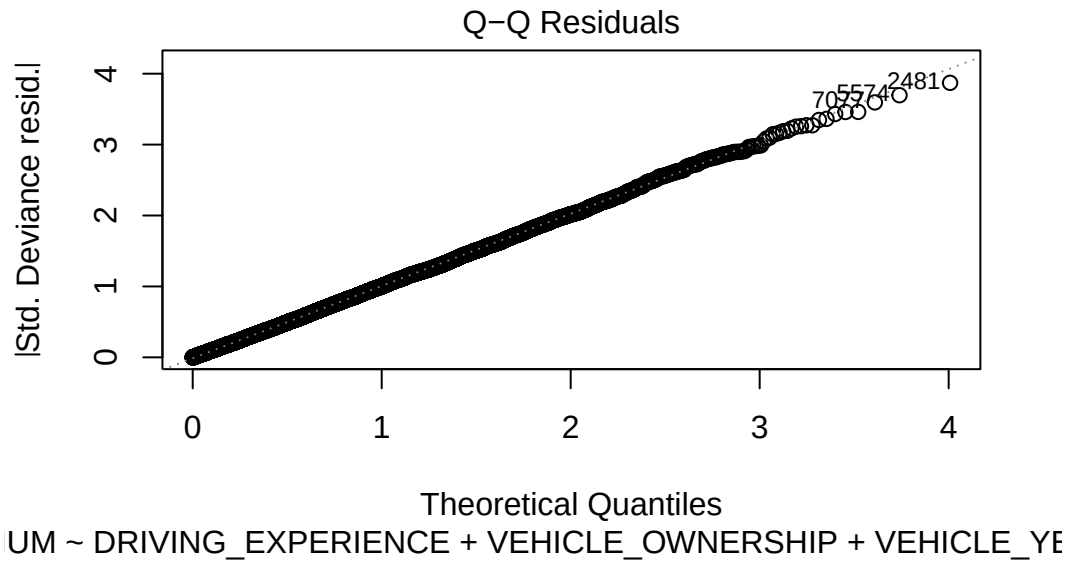
Number of Fisher Scoring iterations: 4

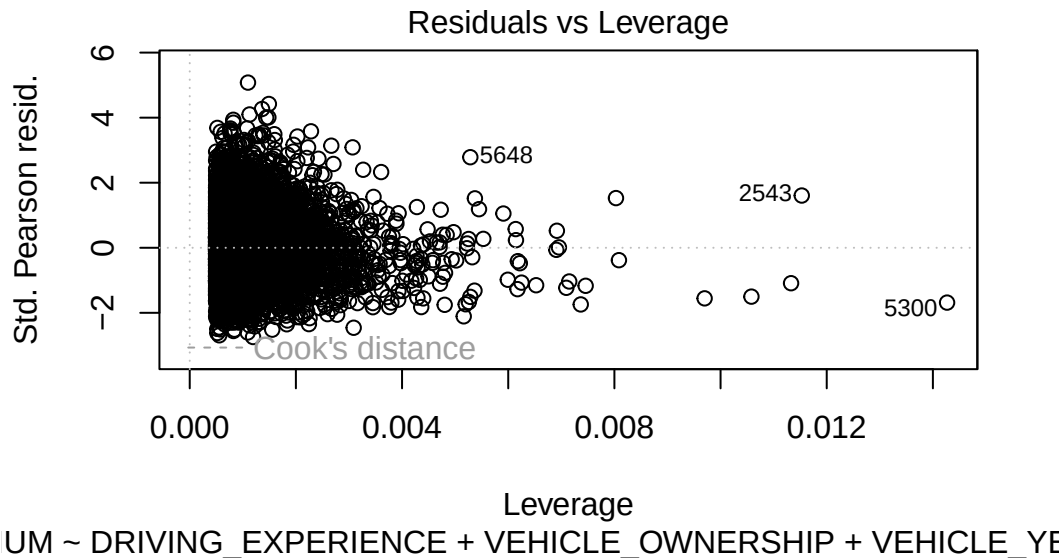
Ha már megvan a modellünk, akkor többféle diagnosztikát is lefuttathatunk a modell jóságának illeszkedésére. A modellszintű illeszkedést a deviancia indikátorok mutatják (lásd később). Itt most az illeszkedést egy másik vetületben értékeljük. Folytonos változók esetén a hibatagokra vonatkozó követelmények hasonlóak, mint a klasszikus lineáris regresszió esetében: függetlenség, nulla várható értékű normális eloszlás. Ezeket legtöbbször vizuálisan is tesztelhetjük, mert ha csak nem sértettünk nagyon komolyan a modell előfeltételein, akkor az optimalizációs kritérium miatt a hibatagokra vonatkozó statisztikai követelmények szinte mindig teljesülnek.

```
plot(backward_gamma)
```



¹a kód a Gemini AI segítségével készült





Látható, hogy a feltételek teljesülnek (elsősorban a QQ ábra vizsgálendő). Az output bevezet még két új mennyiséget:

- Cook-távolság
- Leverage-érték

12.7 Diagnosztikai mutatók

Az alapértelmezett diagnosztika elsősorban a hibatagokat és az illesztett értékeket vizsgálja. Ha a fenti feltételek teljesülnek úgy a befolyásos értékek azonosítása marad a feladat, mivel ezek az értékek torzíthatják a becsléseket. A befolyásos értékeket gyakran *outlier*nek nevezik.

::: {callout-note title="Az outlierokról"} A hagyományos statisztika oktatás során a kiugró, extrém értéket felvevő megfigyeléseknek általában negatív a konnotációja, és az eltávolítását javasolják a mintából. Ennek oka, hogy a klasszikus statisztika mindig az ismert eloszlásokon keresztül közelíti meg az adatokat. Mivel mintáról van szó, valamekkora eltérés a teoretikus eloszlástól tolerálható, ugyanakkor a nagyobb devianciák már valóban torzíthatják a becsléseket, különösen kis minta esetén. A napjainkban használatos nagyobb minták során egy-egy extrém értéket felvevő megfigyelés már tud illeszkedni a teoretikus eloszlás szélére, és nem jelent számottevő torzítást.

Gyakran előfordul azonban, hogy az eloszlás széle vastagabb, vagyis az ott található értékek bár jelentősen távol esnek az átlagtól, de nem számítanak ritka eseménynek. Ilyenkor valóban erős lehet a “húzóerő”, ami egy modell illesztett egyenesét az extrém értékek felé gravitálja. Ez azonban elsősorban csak akkor probléma, ha a függő változó feltételezett eloszlása nincs összhangban a tapasztalati eloszlással. Ilyenkor GLM modellt érdemes alkalmazni, ha egy exponenciális eloszlásra “ráhúzható” a minta eloszlása.

Természetesen előfordulhat, hogy egyetlen ismert eloszlásra sem “húzható rá” a minta eloszlása. Ilyenkor több módszer is adódik:

- Alkalmazzunk robusztus regressziót (GLM esetén is létezik), amely súlyozza a megfigyeléseket, például a *leverage* érték alapján (lásd lentebb).
- Alkalmazzunk zsugorító (*shrinkage*), regularizációs eljárást, amely nem a megfigyelést, hanem a prediktor súlyát szabályozza. Ilyen módszer a [LASSO](#), a [LARS](#), de alkalmazhatók a neurális hálók is. :::

12.7.1 Cook-távolság

A Cook-távolság egy meglepően hatékony mérték annak szemléltetésére, hogy vannak-e olyan outlier megfigyelések, amelyek erősen torzítják a modellt becsült paramétereit. A működési elv a következő:

1. Készítsük el a modellünket, amelybe minden megfigyelést bevonunk.
2. Vegyünk ki egy megfigyelést, majd futtassuk újra modellt.
3. Vizsgáljuk meg, hogyan változtak a becsült együtthatók értékei.
4. Minél nagyobb a változás, annál nagyobb hatást gyakorol a kivett megfigyelés a modellre.

Az elkészült modellen alkalmazzuk a `cook.distance(model)` függvényt, hogy megkapjuk a távolságokat.

12.7.2 Leverage érték

A Cook-távolság hátránya, hogy csak az elkészült modell birtokában lehet alkalmazni, hosszú adatbázisok esetén pedig sokáig tarthat az iteratív művelet, továbbá csak az egyes megfigyelésekre tud információt nyújtani. A *leverage érték* egy lépésben számolható és akár egész változóra is vonatkoztatható, és nem szükséges hozzá a modell számítása. Az eljárás azon az elven alapul, hogy egy vektorban az átlagtól vett távolság alapján egy megfigyelés “*elhúzza*” az illesztett egyenest ettől a várható értéktől. Számítása a következő: legyen $y_i = x_i^T \beta + \epsilon_i$ regressziós modell. Ekkor a *leverage mátrix* a következő:

$$\mathbf{H} = \mathbf{X}(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$$

Ekkor $0 \leq h_{ii} \leq 1$ a *leverage érték (score)*. Minél nagyobb az értéke, annál erősebb befolyással bír az adott megfigyelés. Fontos megjegyezni, hogy a nagy *leverage* érték biztos nagy befolyással bír, de nem feltétlenül outlier.

12.8 Bináris és binomiális függő változók becslése

Legyen $y := \{0, 1\}$ bináris függő változó, és tegyük fel, hogy $E(y|x) = P(y = 1|x) = \pi$, azaz a függő változó értéke meghatározható az x prediktor segítségével. Ez a $\text{var}(y|x) = \pi(1 - \pi)$ variancia struktúrát indukálja. Bináris és binomiális esetben a π valószínűséget közelítjük $\eta = z' \beta$ lineáris prediktorral egy $\pi = h(\eta)$ vonatkozó függvényrel és $g(\pi) = \eta$ link függvénnyel.

A lineáris (normális eloszlást feltételező) GLM modellek alkalmazhatók lennének egy identitás link esetében, azaz $\pi = \eta = z' \beta$ formában. Miután azonban π egy valószínűség, ezért $z' \beta$ lineáris prediktornak a $[0,1]$ tartományon kell maradnia $\forall z$ esetben, ami komoly megkötéseket jelent β lehetséges értékeire nézve. Ezért szükségünk van az alábbi F monoton *eloszlás* függvényre:

$$\pi = F(\eta)$$

F értéke több formát is ölthet, ami meghatározza a következő modelleket is.

12.8.1 Probit modell

A modell általános alakja a következő:

$$\pi = \Phi(\eta) = \Phi(z' \beta)$$

ahol Φ a standard normális eloszlásfüggvény, ami garantálja, hogy π értéke a $[0,1]$ sávban fog maradni.

12.8.2 Logit modell

A logit modell link függvénye:

$$g(\pi) = \log\left(\frac{\pi}{1 - \pi}\right) = \eta$$

A valószínűség becslése a logisztikus eloszlás függvény segítségével történik:

$$\pi = h(\eta) = \frac{\exp(\eta)}{1 + \exp(\eta)}$$

A 0 és 1 közeli értékek kivételével az eloszlás szélein, a *probit* és a *logit* modellek hasonló eredményeket adnak.

Példa Probit és Logit modellek

Használjuk az ismert **car_insur** adatbázist továbbra is, és adjunk becslést arra, hogy lesz-e káreseménye az ügyfélnek! A kalkulált díjat ne használjuk, hiszen ezt a kockázat alapján számoljuk ki.

```
glm(data=car_insur,
     formula = OUTCOME ~.-PREMIUM,
     family = binomial(link = "probit")) -> glm_probit # futtassunk probit modellt

glm(data=car_insur,
     formula = OUTCOME ~.-PREMIUM,
     family = binomial(link = "logit")) -> glm_logit # futassunk logit modellt
```

Hasonlítsuk össze az eredményeket. A probit modell egy eloszlásfüggvény illesztést ad eredménynek, míg a logit modell visszalakítása ennél egyszerűbb.

```
require(ggplot2)
```

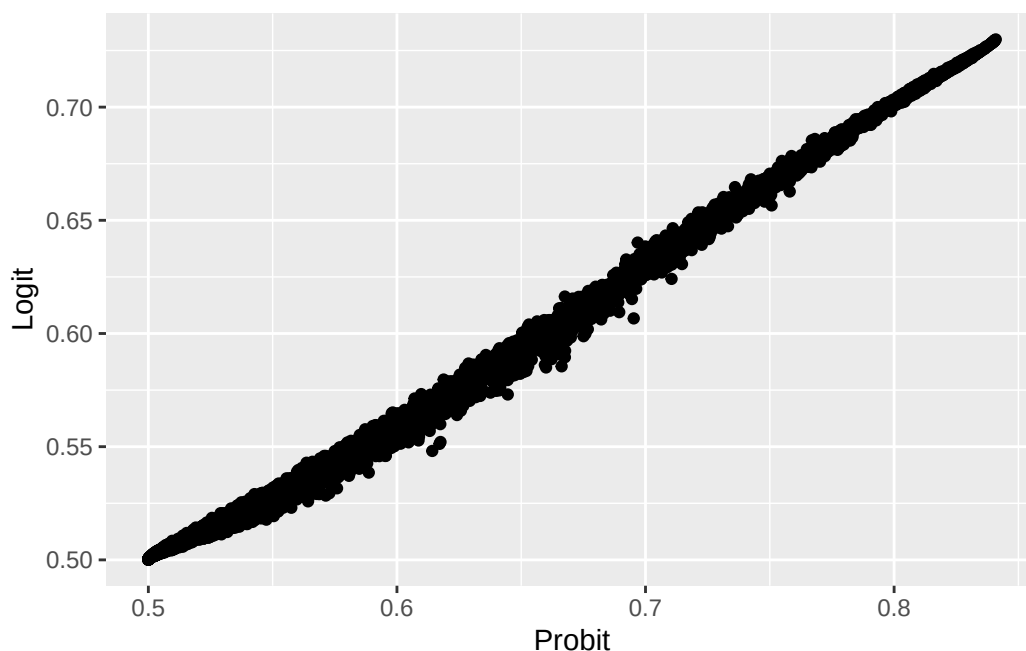
A szükséges csomag betöltődik: ggplot2

```
require(dplyr)

pnorm(glm_probit$fitted.values) -> probit # probit illesztett értékek

exp(glm_logit$fitted.values)/(1+exp(glm_logit$fitted.values)) -> logit # logit illesztet

data.frame(Probit=probit, Logit=logit) %>%
  ggplot(aes(x=Probit, y=Logit)) + geom_point()
```



12.9 Modellek diszkrét eloszlásokra

Amennyiben a probléma természete engedi, úgy a normális eloszlású függő változóra illeszkedő modell is használható (kerekítéssel) vagy, ha csak alacsony értékek fordulnak elő (0, 1, ... q), akkor a multinominális modellek alkalmazandók.

Általánosságban a Poisson eloszlás alapú GLM modellek illeszkednek a legjobban.

12.9.1 Log-lineáris Poisson modell

A legegyszerűbb Poisson GLM, hiszen $\log(\mu) = \eta = z'\beta$ és $\mu = \exp(\eta)$ link függvénnyel kapcsolódik a prediktorhoz. Fontos kikötés azonban, hogy ebben a modellben μ várható érték és a magyarázó változók közötti dependencia *multiplikatív*! Amennyiben additív kapcsolatot sejtünk, hogy a lineáris modellre van szükségünk (azaz a link függvény állítsuk "identity" értékre).

Példa Poisson eloszlású GLM

Illesszünk Poisson modellt az adatokra a **car_insur** adatbázis *PAST_ACCIDENTS* változójára. Itt se használjuk a kikalkulált díjat.

```
glm(data=car_insur,
     PAST_ACCIDENTS~.-PREMIUM,
     family = poisson("log")) -> glm_pois_log # log link függvényrel
```

```
glm(data=car_insur,
     PAST_ACCIDENTS~.-PREMIUM,
     family = poisson("log")) -> glm_pois_log ->glm_pois_ident # identitás link függvény
```

```
summary(glm_pois_log)
```

Call:

```
glm(formula = PAST_ACCIDENTS ~ . - PREMIUM, family = poisson("log"),
     data = car_insur)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-1.817e+01	2.269e+02	-0.080	0.93616
ID	8.744e-09	3.694e-08	0.237	0.81288
AGE26-39	-1.731e-01	3.445e+02	-0.001	0.99960
AGE40-64	-1.942e-01	3.445e+02	-0.001	0.99955
AGE65+	-2.202e-01	3.445e+02	-0.001	0.99949
GENDERmale	7.333e-01	2.484e-02	29.523	< 2e-16 ***
RACEminority	3.840e-02	3.645e-02	1.053	0.29218
DRIVING_EXPERIENCE10-19y	1.922e+01	2.592e+02	0.074	0.94090
DRIVING_EXPERIENCE20-29y	1.993e+01	2.592e+02	0.077	0.93871
DRIVING_EXPERIENCE30y+	2.037e+01	2.592e+02	0.079	0.93737
EDUCATIONnone	-2.401e-02	3.839e-02	-0.626	0.53161
EDUCATIONuniversity	-1.311e-02	2.453e-02	-0.534	0.59304
INCOMEpoverty	-1.926e-01	6.209e-02	-3.103	0.00192 **
INCOMEupper class	2.797e-02	3.305e-02	0.846	0.39739
INCOMeworking class	-1.032e-01	4.423e-02	-2.334	0.01959 *
CREDIT_SCORE	-5.159e-01	1.178e-01	-4.380	1.19e-05 ***
VEHICLE_OWNERSHIP	-5.528e-02	2.870e-02	-1.926	0.05406 .
VEHICLE_YEARbefore 2015	3.224e-02	2.320e-02	1.390	0.16461
MARRIED	3.939e-02	2.643e-02	1.490	0.13614
CHILDREN	4.099e-02	3.205e-02	1.279	0.20092
POSTAL_CODE	-1.541e-05	8.999e-07	-17.119	< 2e-16 ***
ANNUAL_MILEAGE	-6.584e-05	5.077e-06	-12.966	< 2e-16 ***
VEHICLE_TYPESports car	-5.249e-02	5.224e-02	-1.005	0.31498
SPEEDING_VIOLATIONS	-1.557e-02	5.097e-03	-3.054	0.00226 **

```

DUIS                -3.992e-03  1.537e-02  -0.260  0.79501
OUTCOME             -9.149e-02  4.282e-02  -2.137  0.03263 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 17956  on 8148  degrees of freedom
Residual deviance:  6795  on 8123  degrees of freedom
(1851 observations deleted due to missingness)
AIC: 16236

Number of Fisher Scoring iterations: 17

summary(glm_pois_ident)

Call:
glm(formula = PAST_ACCIDENTS ~ . - PREMIUM, family = poisson("log"),
    data = car_insur)

Coefficients:
                Estimate Std. Error z value Pr(>|z|)
(Intercept)    -1.817e+01  2.269e+02  -0.080  0.93616
ID              8.744e-09  3.694e-08   0.237  0.81288
AGE26-39       -1.731e-01  3.445e+02  -0.001  0.99960
AGE40-64       -1.942e-01  3.445e+02  -0.001  0.99955
AGE65+        -2.202e-01  3.445e+02  -0.001  0.99949
GENDERmale     7.333e-01  2.484e-02  29.523 < 2e-16 ***
RACEminority   3.840e-02  3.645e-02   1.053  0.29218
DRIVING_EXPERIENCE10-19y 1.922e+01  2.592e+02   0.074  0.94090
DRIVING_EXPERIENCE20-29y 1.993e+01  2.592e+02   0.077  0.93871
DRIVING_EXPERIENCE30y+  2.037e+01  2.592e+02   0.079  0.93737
EDUCATIONnone  -2.401e-02  3.839e-02  -0.626  0.53161
EDUCATIONuniversity -1.311e-02  2.453e-02  -0.534  0.59304
INCOMEpoverty  -1.926e-01  6.209e-02  -3.103  0.00192 **
INCOMEupper class  2.797e-02  3.305e-02   0.846  0.39739
INCOMeworking class -1.032e-01  4.423e-02  -2.334  0.01959 *
CREDIT_SCORE   -5.159e-01  1.178e-01  -4.380 1.19e-05 ***
VEHICLE_OWNERSHIP -5.528e-02  2.870e-02  -1.926  0.05406 .
VEHICLE_YEARbefore 2015  3.224e-02  2.320e-02   1.390  0.16461

```

```

MARRIED                3.939e-02  2.643e-02   1.490  0.13614
CHILDREN                4.099e-02  3.205e-02   1.279  0.20092
POSTAL_CODE            -1.541e-05  8.999e-07 -17.119 < 2e-16 ***
ANNUAL_MILEAGE         -6.584e-05  5.077e-06 -12.966 < 2e-16 ***
VEHICLE_TYPEsports car -5.249e-02  5.224e-02  -1.005  0.31498
SPEEDING_VIOLATIONS    -1.557e-02  5.097e-03  -3.054  0.00226 **
DUI                    -3.992e-03  1.537e-02  -0.260  0.79501
OUTCOME                -9.149e-02  4.282e-02  -2.137  0.03263 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 17956  on 8148  degrees of freedom
Residual deviance:  6795  on 8123  degrees of freedom
(1851 observations deleted due to missingness)
AIC: 16236

Number of Fisher Scoring iterations: 17

```

12.10 Modell illeszkedés értékelése és a béta koefficiensek tesztelése

A modelleket jellemzően a *skálázott deviancia* segítségével értékeljük:

$$S(y, \hat{\mu}) = 2(\ell(y; \phi, y) - \ell(\hat{\mu}, \phi, y))$$

Azaz annak a log-likelihoodnak a különbsége, hogy a feltételezett paraméterek mellett a megfigyelt adatsort, illetve a becsült adatsort kapjuk.

Egy másik indikátor *reziduális deviancia*, ami tulajdonképpen a skálázott deviancia skála-paraméterrel korrigált változata:

$$D(y, \hat{\mu}) = \phi S(y, \hat{\mu})$$

Két modell összehasonlítására a *likelihood arány próba* szolgál. Legyen Model1 $\eta = X\beta$ alakú, Model2 pedig $\eta = X\beta + Z\gamma$ alakú, ahol $\text{rank}(Z) = r$. Model1 dimenziószáma p_1 , Model2 dimenziószáma pedig $p_2 = p_1 + r$. Más szavakkal Model1 egy szűkebb, Model2 pedig egy tágabb modell. A teszt statisztika a következő:

$$2(\ell_2 - \ell_1) = S(y; \hat{\mu}_1) - S(y; \hat{\mu}_2) = \frac{D(y; \hat{\mu}_1) - D(y; \hat{\mu}_2)}{\hat{\phi}}$$

Ha ϕ ismert, akkor a teszthez tartozó eloszlás χ^2 eloszlást követ. Ha ϕ nem ismert, akkor lehetséges a Model2 által becsült $\hat{\phi}$ értéket használni r dimenziószámmal korrigálva, ekkor azonban F -próbát kell alkalmaznunk:

$$\frac{D(y; \hat{\mu}_1) - D(y; \hat{\mu}_2)}{\hat{\phi}r} \sim F_{r, n-p_2}$$

A GLM modellek outputjában megtalálható a *Null deviance* mutató, ami a teljes (szaturált, minden magyarázó változót tartalmazó) modell hasonlítja a null-modellhez, ami csak egy paraméter becslést tartalmaz (jellemzően a várható értéket).

Értelemszerűen, ha a null devianciánk nagyon alacsony, akkor a teljes (szaturált) modellünk nem sokkal jobb, mint a null-modellünk (analógiájában a lineáris regressziók esetében alkalmazott globális F -próbához hasonló, csak hipotézis vizsgálat nélkül).

A β koeficiensre vonatkozó t -próbák a szokásos $t = \frac{\hat{\beta} - \beta}{\sigma_\beta}$ módon történik.

Két modell összehasonlításához használható még az *Akaike információs kritérium* is:

$$AIC(\hat{\mu}) = D(y; \hat{\mu}) + 2p\phi$$

ahol p a modell paramétereinek száma (a modell dimenziószáma).

12.11 Túlszóródás

A legtöbb lineáris regressziós modellben, mikor $y \in \mathbb{R}$, feltételezzük, hogy $\epsilon \sim \mathcal{N}(0, \sigma^2)$. A modellben mind a várható érték, mind a variancia független egymástól, hiszen $E(Y|x) = \mu(x)$ és $Var(Y|x) = \sigma^2$ (ami nem függ x -től).

Poisson vagy logisztikus regressziós GLM modellekben a várható érték és a variancia nem független egymástól. Poisson esetben ismert, hogy $E(Y|x) = \mu(x)$ és $Var(Y|x) = \mu(x)$.

Logisztikus regresszió esetén pedig legyen például a függő változónk n_m darab próba, aminek várható értéke: $E(Y|x) = \mu(x) = n_m \pi(x)$. Ilyen esetben ismert, hogy $Var(Y|x) = n_m \pi(x)(1 - \pi(x))$, vagyis a fenti két esetben a várható érték és a variancia tényleg nem független egymástól, amit a modellben kezelni kell.

Regressziós modellekben természetesen gyakran előfordul, hogy a variancia viselkedése nem felel meg az előzetes várakozásainknak, más szavakkal a σ^2 nem konstans minden x érték esetében. A jelenség *heteroszkedaszticitás* néven is ismert. GLM-ek esetében tipikusan ilyen

helyzet áll elő, ha valaminek az elemszámát (“*counts*”) vagy a megoszlását modellezzük. A modellek rosszul illeszkednek annak ellenére, hogy a becült várható érték (vagy azzal ekvivalens paraméter) egyébként jól lett közelítve. Ezt a jelenséget **túlszóródásnak** (“*overdispersion*”) nevezik.

12.11.1 A túlszóródás okai

Lineáris regressziós modellek esetén mikor beveszünk egy fontosnak tartott magyarázó változót a modellbe, akkor az általa megmagyarázott variancia “kikerül” a hiba négyzetösszegből és a megmagyarázott négyzetösszegbe “kerül”. Megfordítva a dolgot, ha egy fontos változót kihagyunk a modellből, akkor az *SSE* érték nagyobb lesz, mintha a változó a modellben szerepelne. Következésképp az átlagos négyzetes hiba (*MSE*) elinflálódik, és a valós σ^2 variancia felül lesz becsülve.

Ezzel szembe, ha binomiális vagy Poisson modellben hagyunk ki egy fontos magyarázó változót, akkor a modell egyes megfigyelések becült értékeire ugyanazt a várható értéket adja, holott azok tényleges értéke eltérő a kihagyott változó hatása miatt.

A kihagyott változón kívül még a túlszóródás oka lehet, hogy a standard binomiális és Poisson modellek feltételezik, hogy a megfigyelések vagy a próbák és kísérletek egymástól függetlenek. Tipikusan ez az eset állhat elő, ha a mintavétel klaszterekből, csoportosan történik. Például ha egy gyártósorról veszünk mintát, akkor a hibás elemek nagy valószínűséggel hasonló mintázatot fognak követni, mintha különböző, véletlenszerű gyártósorokról vennénk őket. Ez a pozitív korreláció nagyobb varianciát eredményez a modellekben a túlillesztés miatt.

12.11.2 Kvázi-likelihood modellek

Sok esetben a túlszóródásnak könnyen kezelhető oka van, például kiugró értékek. Ugyanakkor, ha a jelenséget egy kihagyott változó okozza és ezt nem tudjuk orvosolni, akkor más megoldáshoz kell folyamodni. Ilyen lehet például, hogy a egy másik eloszláscsaládot használunk fel a modellezés során egy olyan extra paraméterrel, ami a nem megmagyarázott varianciát méri.

A kvázi-likelihood modellek binomiális és Poisson esetekben használhatók. Legyen például egy $y \sim \text{Pois}(\mu(x))$ függő változónk $\mu(x)$ várható értékkel, ahol x a magyarázó változókat jelöli. Tegyük fel, hogy a túlszóródás a variancia konstans inflálódásából fakad, azaz $\text{Var}(y|x) = \gamma\mu(x)$, ahol $\gamma \geq 1$ egy diszperziós paraméter. A kvázi-likelihood modell úgy alkotható meg ebben az esetben, ha a Poisson log-likelihood modellt elosztjuk γ paraméterrel. Ez nem változtatja meg a becült paramétereket. γ paraméterre a becslést a *Pearson-féle illeszkedési statisztika* és *hibatagok szabadságfokának hányadosa* adja.

Kvázi-poisson regresszió

Az eredeti példa [itt](#) található. Mivel a túlszóródást csak ott tudjuk megmutatni, ahol valóban jelen van, ezért olyan adatbázist választottunk, ahol biztosan jól megmutatható a módszer.

Ebben a kutatásban azt vizsgálták, hogy egy házasságban hányszor lép félre az egyik fél. A félrelépések száma valószínűleg Poisson eloszlást követ.

```
require(AER)
```

A szükséges csomag betöltődik: AER

Warning: a(z) 'AER' csomag az R 4.6.1 verziójával lett fordítva

A szükséges csomag betöltődik: car

Warning: a(z) 'car' csomag az R 4.6.1 verziójával lett fordítva

A szükséges csomag betöltődik: carData

Warning: a(z) 'carData' csomag az R 4.6.1 verziójával lett fordítva

Kapcsolódás csomaghoz: 'car'

The following object is masked from 'package:dplyr':

```
recode
```

A szükséges csomag betöltődik: lmtest

Warning: a(z) 'lmtest' csomag az R 4.6.1 verziójával lett fordítva

A szükséges csomag betöltődik: zoo

Kapcsolódás csomaghoz: 'zoo'

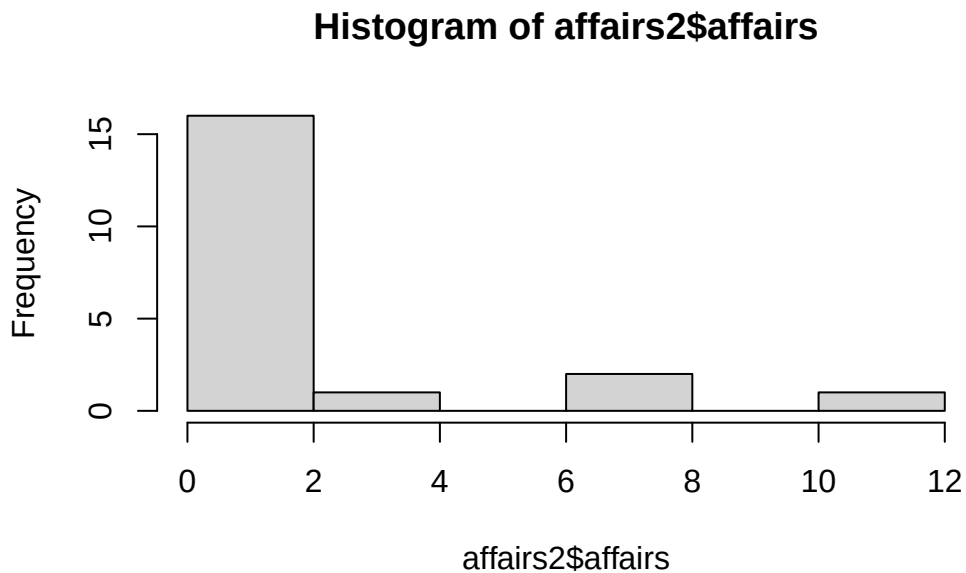
The following objects are masked from 'package:base':

```
as.Date, as.Date.numeric
```

A szükséges csomag betöltődik: sandwich

Warning: a(z) 'sandwich' csomag az R 4.6.1 verziójával lett fordítva

```
require(dplyr)
data(Affairs)
affairs2<-sample_n(Affairs, 20)
hist(affairs2$affairs)
```



Feltételezésünk valószínűleg megállja a helyét. Futtassuk le a GLM regressziókat.

```
pois.model<-glm(data=affairs2, affairs~., family = "poisson")
```

Warning: glm.fit: fitted rates numerically 0 occurred

```
summary(pois.model)
```

Call:

```
glm(formula = affairs ~ ., family = "poisson", data = affairs2)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	154.520	559446.633	0	1
gendermale	-9.108	53823.217	0	1
age	-2.964	12183.862	0	1
yearsmarried	2.704	13502.705	0	1
childrenyes	31.247	179813.683	0	1
religiousness	-6.689	73758.865	0	1
education	-3.398	29045.198	0	1
occupation	1.617	43736.516	0	1
rating	-13.315	46297.982	0	1

(Dispersion parameter for poisson family taken to be 1)

Null deviance: 9.6387e+01 on 19 degrees of freedom
 Residual deviance: 3.2305e-10 on 11 degrees of freedom
 AIC: 34.944

Number of Fisher Scoring iterations: 25

Látjuk, hogy a diszperziós paraméter a modellben $\gamma = 1$. Ellenőrizzük, hogy ez valóban így van-e. A Pearson-féle illeszkedési statisztika rendkívül egyszerű:

$$X^2 = \sum_{i=1}^n \frac{(y_i - \tilde{\lambda}_i)^2}{\tilde{\lambda}_i}$$

```
pois.person<-residuals(pois.model, type = "pearson")
```

Számoljuk ki a becsült diszperziós paramétert, ami a person-féle statisztika a modell hibáinak szabadságfoka.

```
sum(pois.person^2)/pois.model$df.residual
```

```
[1] 1.468431e-11
```

Az egységnyi diszperziós paraméterre tett feltételezést valószínűleg elvethetjük. Futtassunk kvázipoisson modellt.

```
quasi.pois.model<-glm(data=affairs2, affairs~., family = "quasipoisson")
summary(quasi.pois.model)
```

```

Call:
glm(formula = affairs ~ ., family = "quasipoisson", data = affairs2)

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  154.52046    3.53425  43.721 1.09e-13 ***
gendermale   -9.10815    0.34002 -26.787 2.28e-11 ***
age          -2.96420    0.07697 -38.511 4.38e-13 ***
yearsmarried  2.70417    0.08530  31.701 3.66e-12 ***
childrenyes  31.24695    1.13596  27.507 1.71e-11 ***
religiousness -6.68942    0.46596 -14.356 1.81e-08 ***
education    -3.39846    0.18349 -18.521 1.22e-09 ***
occupation    1.61733    0.27630   5.853 0.00011 ***
rating       -13.31504    0.29248 -45.524 7.03e-14 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for quasipoisson family taken to be 3.990965e-11)

Null deviance: 9.6387e+01  on 19  degrees of freedom
Residual deviance: 3.2305e-10  on 11  degrees of freedom
AIC: NA

Number of Fisher Scoring iterations: 25

```

A kvázi-poisson modellben a változók szignifikancia szintje “elveszhet”, mivel a konfidencia intervallum kiszélesedik, kompenzálандó a túlszóródást, hiszen egy Poisson modellben a $\mu = \sigma^2$. Ezzel ugyan úgy tűnhet, hogy egy rosszabb modellre cseréltük az eredeti regressziókat, azonban éppen ez a “reális”, hiszen egy rosszul illeszkedő modellünk volt korábban is szignifikáns változókkal.

Vegyük észre, hogy az AIC mutató nem definiált, ennek oka, hogy a likelihood függvény a kvázi-likelihood modellek esetében nem jól definiált.

Fontos, hogy a kvázi-likelihood modelleket csak abban az esetben használjuk, ha valóban arra gyanakszunk, hogy a túlszóródást kihagyott változók okozzák!